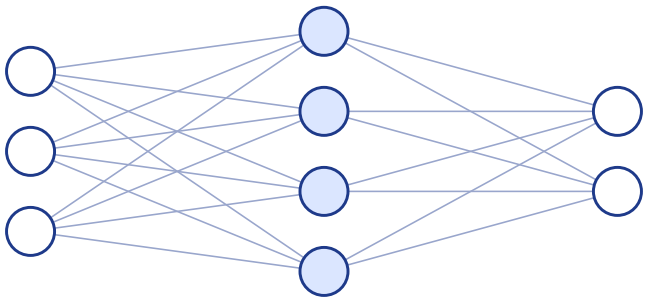
MA4144
Neural Networks and Fuzzy Logic

Study Notes - Part 1: Neural Networks



University of Moratuwa
Semester 7

Lessons 1–5

Introduction · McCulloch–Pitts neuron and perceptron · ADALINE and MADALINE
Widrow–Hoff (LMS) learning · Multilayer perceptrons and backpropagation · Probabilistic neural networks

Neurons, learning rules and network topologies, with the derivations and worked examples behind each one.

Introduction to Neural Networks

1 Introduction

Artificial Neural Networks (ANNs), also called **Neural Networks**, are computational models inspired by the structure and functioning of the human brain. They are designed to process information using many interconnected simple processing elements called **neurons**. Neural networks are used in pattern recognition, classification, prediction, function approximation, and intelligent decision-making.

The fundamental idea is that intelligence can emerge from the cooperation of many simple computational units. Each neuron receives inputs, performs a mathematical operation, and produces an output. By adjusting the strengths of the connections between neurons, the network gradually improves its performance and learns patterns from examples.

2 What is a Neural Network?

A neural network is a system made up of interconnected neurons that work together to solve a problem. It can be viewed as a generalized mathematical model inspired by biological neural systems and human cognition. In simple terms, it is a machine learning model that transforms input data into output predictions through layered computation.

A neural network is built on the following assumptions:

(a) Information processing occurs at many simple elements called neurons.

(b) Signals are passed between neurons over connection links.

(c) Each connection link has an associated **weight**, which multiplies the transmitted signal.

(d) Each neuron applies an **activation function** to its net input to determine its output.

These assumptions form the basic operating principle of all artificial neural networks.

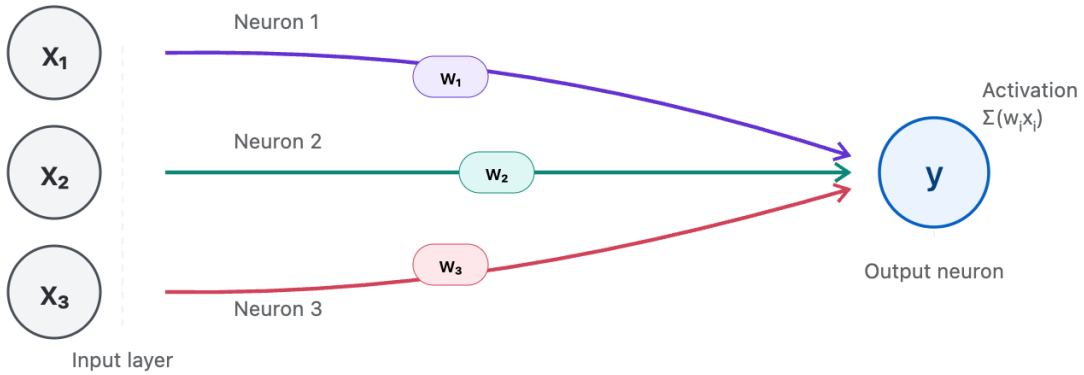


Figure 1: Single Neuron Model

3 Characteristics of a Neural Network

A neural network is characterized by three major features:

- **Architecture**: Architecture refers to the arrangement of neurons into layers and the pattern of connections between them. It determines how information flows through the network.
- **Learning Algorithm**: The learning algorithm is the method used to adjust the weights on the connections. It is also called the **training algorithm**.
- **Activation Function**: The activation function is the mathematical rule that transforms the net input into the neuron's output. It introduces flexibility and nonlinearity into the model.

Together, architecture, learning algorithm, and activation function determine how effective a network is for a given task.

3.1 Architecture of Neural Networks

The arrangement of neurons into layers and the connection pattern between them is called the **network architecture**. A neural network usually contains:

(a) **Input Layer**: receives the raw data,

(b) **Hidden Layer(s)**: carry out internal processing and feature extraction,

(c) **Output Layer**: produces the final prediction or decision.

Hidden layers enable neural networks to automatically learn nonlinear combinations of features. When a classification problem cannot be separated by a straight line, a network with hidden layers can learn transformed representations that make the classes separable. Therefore, hidden layers are valuable because they:

(a) detect intermediate patterns,

(b) build more abstract representations from raw inputs, and

(c) allow the network to solve nonlinear problems.

Neural networks can be classified into two main categories:

- **Single-Layer Network**: A single-layer network has one layer of trainable weights connecting the input layer directly to the output layer.

  Main features:

  (a) no hidden layer,

  (b) direct input-output connection,

  (c) suitable only for simple linearly separable problems.

- **Multilayer Network**: A multilayer network contains one or more hidden layers between input and output.

  Main features:

  (a) can represent complex decision boundaries,

  (b) hidden layers extract intermediate features,

  (c) can approximate a wide class of functions.

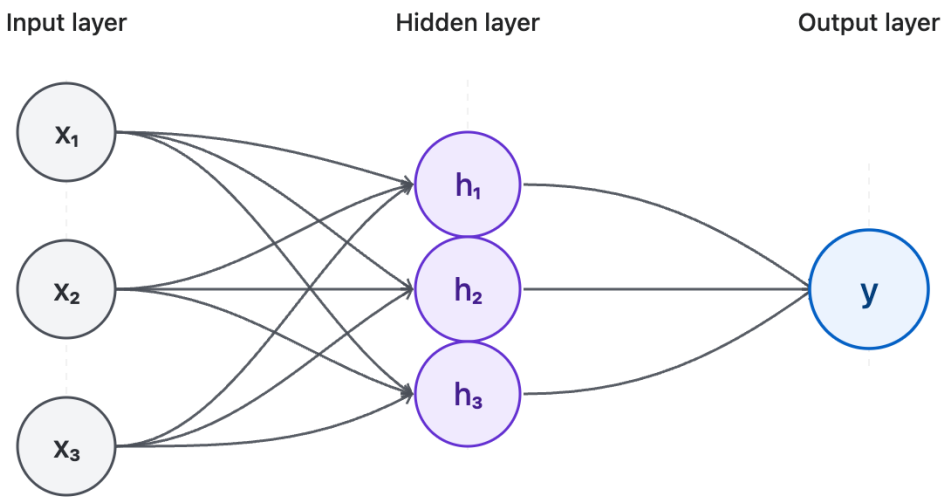


Figure 2: Multilayer Network

3.2 Learning in Neural Networks

Learning is the process through which a neural network adjusts its weights in order to improve its performance. The objective is to make the network's output as close as possible to the desired or useful response for a given input.

During training, the learning process typically involves the following steps:

(a) **Forward pass**: the input is propagated through the network, and each layer computes its output.

(b) **Prediction**: the network produces a final output.

(c) **Loss calculation**: the predicted output is compared with the true value using a loss function.

(d) **Backward pass (backpropagation)**: the error is propagated backward through the network.

(e) **Weight update**: an optimization algorithm, such as gradient descent, updates the weights and biases.

By repeating this process over many training examples, the network gradually learns from data.

3.2.1 Supervised Learning

In supervised learning, the network is trained using input-output pairs. The process is as follows:

(a) present an input vector to the network,

(b) provide the desired target output,

(c) compute the actual output,

(d) measure the difference between the actual and target outputs,

(e) adjust the weights to reduce the error.

This approach is used when the correct outputs are already known.

One of the most important supervised learning algorithms used in neural network training is **backpropagation with gradient descent**. It computes how the error at the output depends on each weight in the network by applying the chain rule of calculus. Once these error contributions are determined, the weights can be adjusted to reduce the overall loss.

Gradient descent is the optimization method most commonly used for this purpose. It updates the model parameters step by step, gradually moving the network toward lower error.

3.2.2 Unsupervised Learning

In unsupervised learning, only input vectors are provided.

(a) no target output is available,

(b) the network identifies patterns, groupings, or structures in the data,

(c) learning is based on similarities and the internal organization of the data.

This approach is useful for clustering, feature discovery, and data compression.

Table 1: Comparison Between Supervised and Unsupervised Learning

Feature	**Supervised Learning**	**Unsupervised Learning**
Training data	Input with target output	Input only
Target output	Present	Absent
Goal	Learn an input-output mapping	Discover structure or patterns
Common use	Classification, prediction	Clustering, grouping

3.3 Activation Functions

The activation function determines the neuron's output based on its net input. Its main role is to introduce **nonlinearity**. Without nonlinear activation, even a multilayer network would behave like a simple linear model.
Common purposes of activation functions include:

(a) **Deciding whether a neuron should be active:** The activation function determines how strongly a neuron responds to its input. If the net input is too small, the neuron may produce little or no output; if it is large enough, the neuron becomes more active. In this way, the activation function controls whether the information passing through that neuron is important enough to be forwarded to the next layer.

(b) **Limiting the output range:** Activation functions often restrict the output of a neuron to a specific range, such as between 0 and 1 or between -1 and 1. This helps keep the values within manageable limits, makes the network more stable during training, and can also make the outputs easier to interpret.

(c) **Enabling the network to model nonlinear relationships:** This is the most important role of activation functions. Many real-world problems are nonlinear, meaning the relationship between input and output cannot be represented by a straight line. By introducing nonlinearity, activation functions allow the network to learn complex patterns, curved decision boundaries, and hidden feature combinations that a purely linear model cannot capture.

A few commonly used activation functions are presented below.

Step Function

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

This function is simple and useful in basic threshold models.

Sigmoid Function

$$f(x) = \frac{1}{1 + e^{-x}}$$

Properties:

(a) output lies between 0 and 1,

(b) smooth and differentiable,

(c) useful in certain learning algorithms and probabilistic interpretation.

Hyperbolic Tangent Function

$$f(x) = \tanh(x)$$

Properties:

(a) output lies between -1 and 1,

(b) often preferred over sigmoid in some settings due to symmetry around zero.

Linear Function

$$f(x) = x$$

This is commonly used in output layers for regression problems where continuous numerical output is required.

ReLU Function

The Rectified Linear Unit (ReLU), widely used in modern deep learning:

$$f(x) = \max(0, x)$$

Properties:

(a) simple and computationally efficient,

(b) helps deep networks train effectively,

(c) widely used in hidden layers.

Table 2: Common transfer functions and their MATLAB representations

Name	**Input/Output Relation**	**MATLAB Function**
Hard Limit	$a = 0 \quad n < 0$ $a = 1 \quad n \geq 0$	`hardlim`
Symmetrical Hard Limit	$a = -1 \quad n < 0$ $a = +1 \quad n \geq 0$	`hardlims`
Linear	$a = n$	`purelin`
Saturating Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n \leq 1$ $a = 1 \quad n > 1$	`satlin`
Symmetric Saturating Linear	$a = -1 \quad n < -1$ $a = n \quad -1 \leq n \leq 1$ $a = 1 \quad n > 1$	`satlins`
Log-Sigmoid	$a = \frac{1}{1+e^{-n}}$	`logsig`
Hyperbolic Tangent Sigmoid	$a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$	`tansig`
Positive Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n$	`poslin`
Competitive	$a = 1$ neuron with max n $a = 0$ all other neurons	`compet`

4 Mathematical Model of a Neuron

The mathematical model of a neuron describes how a neuron takes several input values, combines them, and produces one output.
Consider a neuron receiving inputs

$$x_1, x_2, x_3, \ldots, x_n$$

with corresponding weights

$$w_1, w_2, w_3, \ldots, w_n.$$

The **net input** to the neuron is the weighted sum of the input values:

$$y_{in} = \sum_{i=1}^{n} w_i x_i$$

If a bias term b is included, then:

$$y_{in} = b + \sum_{i=1}^{n} w_i x_i$$

The final output is obtained by applying an activation function f to the net input:

$$y = f(y_{in})$$

Note that

- **Input** (x_i)**:** the signal or information entering the neuron.
- **Weight** (w_i)**:** the parameter that controls the influence of each input on the neuron.

  (a) A large positive weight increases the importance of an input.

  (b) A large negative weight decreases or reverses the effect of an input.

  (c) A weight close to zero means that the input has very little influence.

  During training, the network adjusts these weights to improve performance.

- **Bias** (b)**:** a constant term that shifts the activation function (activation curve is moved horizontally) and helps set the neuron's threshold.

  The bias can be understood as:

  (a) an additional adjustable parameter,

  (b) a constant input of value 1 with its own weight,

  (c) a mechanism that improves representational power.

  Without a bias term, the neuron may become too restricted to represent useful patterns.

4.1 Matrix Representation

For compact mathematical expression, neuron computation can be written in vector form.

Let

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \qquad w = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

Then:

$$y_{in} = w^T x + b$$

and

$$y = f(w^T x + b)$$

This notation is useful for:

(a) compact representation,

(b) efficient computer implementation,

(c) extending from a single neuron to a full layer or network.

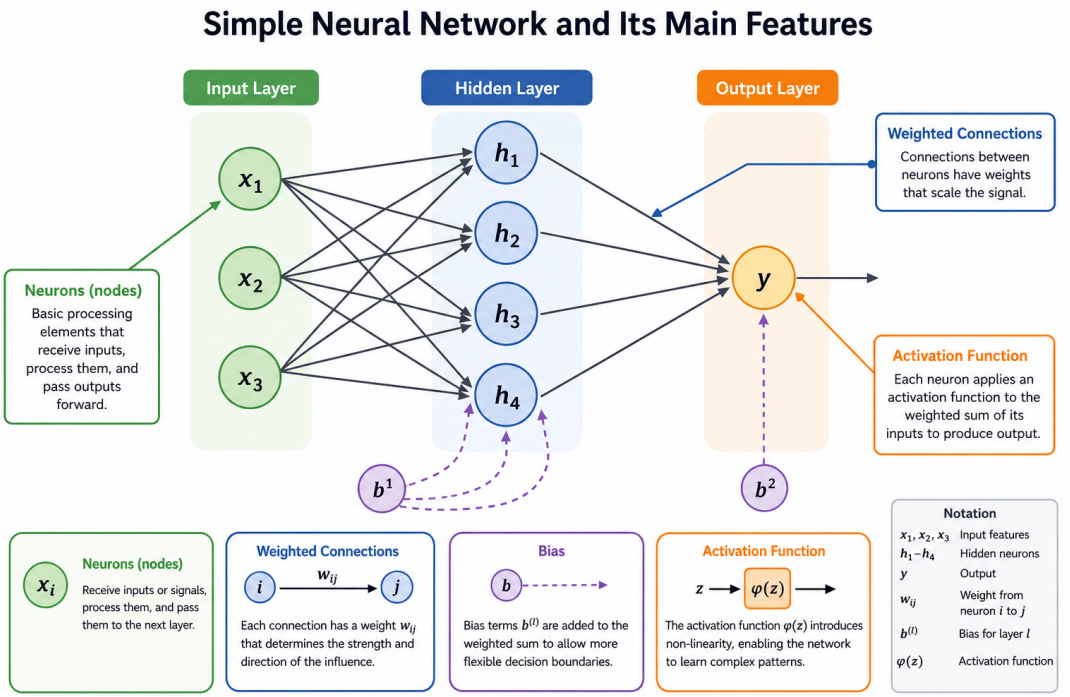


Figure 3: Architecture of a simple Neural Network

5 Learning Rate and Threshold

5.1 Learning Rate

The **learning rate**, usually denoted by α, determines how much the weights change during training.

(a) If α is too large, learning may be unstable.

(b) If α is too small, learning may be too slow.

Hence, choosing an appropriate learning rate is essential.

5.2 Threshold

The threshold determines the level at which a neuron becomes active. In some formulations, the threshold can be absorbed into the bias term for simpler computation.

Table 3: Important Notation

Symbol	**Meaning**
x_i	input activation of unit i
y_j	output activation of unit j
w_{ij}	weight from unit i to unit j
W	weight matrix
α	learning rate
θ	threshold
s	input training vector
t	target output vector
b	bias term
f	activation function

Example 1. *Let the inputs to a neuron be*

$$x_1 = 1, \quad x_2 = 2, \quad x_3 = -1,$$

with corresponding weights

$$w_1 = 0.5, \quad w_2 = -1, \quad w_3 = 2,$$

and bias

$$b = 1.$$

The net input is given by

$$\begin{aligned} y_{in} &= b + \sum_{i=1}^{3} w_i x_i \\ &= 1 + (0.5)(1) + (-1)(2) + (2)(-1) \\ &= -2.5 \end{aligned}$$

Applying the step activation function,

$$y = \begin{cases} 1, & y_{in} \geq 0 \\ 0, & y_{in} < 0 \end{cases}$$

Since the net input is negative,

$$y_{in} = -2.5 < 0,$$

the neuron produces the output

$$y = 0.$$

□

Example 2. *Consider a neuron with output*

$$y = f(x + b)$$

where f is an activation function and b is the bias. Let the activation function be the sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Then the neuron output becomes

$$y = \frac{1}{1 + e^{-(x+b)}}$$

Case 1: No bias

If $b = 0$, then

$$y = \frac{1}{1 + e^{-x}}$$

The midpoint of the sigmoid occurs at $x = 0$.

Case 2: Positive bias

If $b = 2$, then

$$y = \frac{1}{1 + e^{-(x+2)}}$$

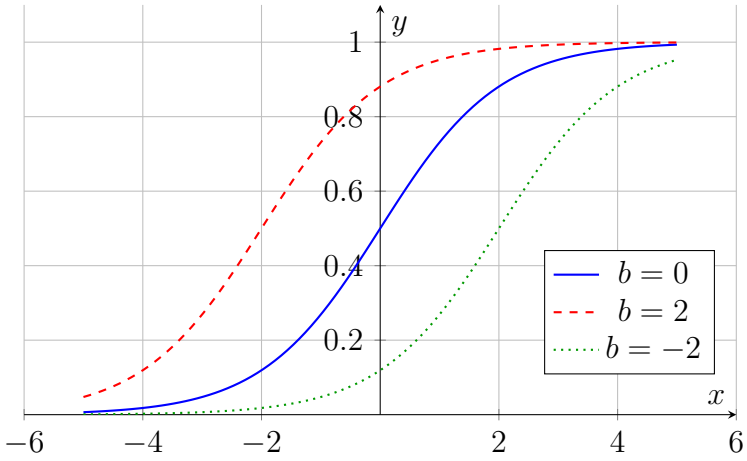
*Now the midpoint occurs at $x = -2$. So the graph shifts **2 units to the left**.*

Case 3: Negative bias

If $b = -2$, then

$$y = \frac{1}{1 + e^{-(x-2)}}$$

*Now the midpoint occurs at $x = 2$. So the graph shifts **2 units to the right**.*



Thus,

- *A positive bias shifts the activation function to the **left**.*
- *A negative bias shifts the activation function to the **right**.*

□

6 Importance and Advantages of Neural Networks

Neural networks are important because many real-world problems are **nonlinear**, and simple linear models cannot always represent complex patterns in data. One of their greatest strengths is that they reduce the need for manual feature engineering by automatically learning useful combinations of input features during training. Through hidden layers and nonlinear activation functions, neural networks transform raw input data into increasingly meaningful internal representations. These learned representations enable the network to capture complex nonlinear relationships and make accurate predictions.

The main advantages of neural networks are:

(a) they learn from examples,

(b) they can model complex and nonlinear relationships,

(c) they are useful for pattern recognition and classification,

(d) they can generalize from training data to unseen data,

(e) they are adaptable and improve with training,

(f) they reduce the need for manual feature engineering by automatically learning useful feature combinations.

Because of these strengths, neural networks are widely used in tasks such as:

(a) image recognition,

(b) speech recognition,

(c) natural language processing,

(d) forecasting,

(e) recommendation systems,

(f) anomaly and fraud detection.

7 Limitations of Neural Networks

The main limitations are:

(a) large amounts of training data may be required,

(b) training may be computationally expensive,

(c) internal decision-making can be difficult to interpret,

(d) performance depends strongly on architecture, activation function, and training method,

(e) overfitting may occur when the network becomes too complex relative to the data.

8 Deep Neural Networks, Modern Extensions, and Applications

Deep neural networks contain many hidden layers, which enable them to learn increasingly abstract features from data. In general, early layers detect simple patterns, middle layers combine them into more complex structures, and later layers form highly abstract representations that are useful for final prediction. This ability to automatically extract meaningful features makes deep neural networks much more powerful than simple single-layer models.

Modern neural networks include several important types, each designed for particular kinds of data and tasks:

(a) **Convolutional Neural Networks (CNNs)** for image-related tasks,

(b) **Recurrent Neural Networks (RNNs)** for sequential and time-dependent data,

(c) **Transformers** for modern language and sequence processing.

Because of their ability to learn from complex data and automatically extract useful representations, neural networks are widely used in many applications, including:

(a) **Computer vision**: image classification, facial recognition, and medical imaging.

(b) **Natural language processing**: machine translation, text generation, summarization, and chatbots.

(c) **Speech recognition**: voice assistants and transcription systems.

(d) **Prediction and forecasting**: weather prediction, financial trends, and demand estimation.

(e) **Pattern recognition and anomaly detection**: fraud detection, fault diagnosis, and document classification.

McCulloch–Pitts Neuron Model and Perceptron Logic

1 McCulloch–Pitts Neuron Model

The McCulloch–Pitts neuron model was proposed in **1943** by **Warren McCulloch**, a neuroscientist, and **Walter Pitts**, a logician. Their goal was to describe how a neuron could be represented as a simple computational unit. In this model, the neuron receives binary inputs, combines them, compares the total with a fixed threshold, and produces a binary output. Therefore, the McCulloch–Pitts neuron behaves like a **threshold logic unit** that can either fire or remain inactive.

This model was important because it showed that neural activity could be interpreted in terms of **logic and computation**. Using suitable thresholds and connections, McCulloch–Pitts neurons can implement simple logical operations such as **AND, OR, and NOT**. For this reason, the model is considered the first formal artificial neuron model and a major milestone in the development of neural network theory.

However, the McCulloch–Pitts model has an important limitation: it does not have a learning mechanism. Its weights and threshold must be fixed in advance, which means it can perform logic functions but cannot adapt itself from examples. To overcome this limitation, later models introduced adjustable weights and learning rules.

1.1 Basic Idea

The McCulloch–Pitts (M-P) neuron is a **binary threshold unit**. It receives binary inputs, sums them according to fixed connection weights, and produces a binary output:

$$y = \begin{cases} 1, & \text{if the neuron fires} \\ 0, & \text{otherwise} \end{cases}$$

Thus, the neuron behaves like a logical device. [1]

1.2 Requirements for McCulloch–Pitts Neuron

The requirements for McCulloch–Pitts neuron may be summarized as follows:

1. The activation of McCulloch–Pitts neuron is **binary**.
2. McCulloch–Pitts neurons are connected by **directed, weighted paths**.

[1] If the neuron fires, it means that the total input has reached or exceeded the threshold, and therefore the neuron produces an output of 1. Otherwise, the output is 0.

3. A connection is **excitatory**[2] if the weight on the path is positive; otherwise, it is **inhibitory**. All excitatory connections into a particular neuron have the same weights.

4. Each neuron has a **fixed threshold** such that if the net input to neuron is greater than or equal to the threshold, the neuron fires.

5. The threshold is set so that **inhibition is absolute**. That is, any nonzero inhibitory input will prevent the neuron from firing.

6. It takes **one time step** for a signal to pass over one connection link.

1.3 Architecture

An M-P neuron Y may receive signals from any number of other neurons. Each connection path is either:

- **excitatory**, with weight $w > 0$, or
- **inhibitory**, with weight $-p$, where $p > 0$.

The activation function for unit Y is

$$f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} \geq \theta \\ 0, & \text{if } y_{in} < \theta \end{cases} \tag{1}$$

where:

- y_{in} is the total input signal received,
- θ is the threshold.

Threshold Condition for Absolute Inhibition

The condition that inhibition is absolute requires that θ satisfy the inequality

$$\theta > nw - p \tag{2}$$

where:

- n is the number of excitatory inputs,
- w is the excitatory weight,
- p is the magnitude of the inhibitory weight.

This condition ensures that even if all excitatory inputs are active, one inhibitory input is sufficient to stop the neuron from firing.

[2] An excitatory input is a positive input signal that increases the total activation of a neuron and helps it reach the firing threshold.

Condition for Firing with k Excitatory Inputs

Let k be the minimum number of active excitatory inputs required to trigger the neuron.

When exactly k inputs are active, the total input energy is kw. Because k is the minimum number of inputs required to fire, this total energy must be greater than or equal to the threshold:

$$kw \geq \theta$$

Conversely, if only $k-1$ inputs are active, the neuron must not fire. Therefore, the total input energy $(k-1)w$ must be strictly less than the threshold:

$$\theta > (k-1)w$$

Combining them gives the full condition for a neuron that requires at least k inputs to fire:

$$kw \geq \theta > (k-1)w \tag{3}$$

This means:

- if fewer than k excitatory inputs are active, the neuron does not fire,
- if k or more excitatory inputs are active, the neuron fires.

Remark. Although all excitatory weights coming into any particular unit must be the same, the weights coming into one unit, say Y_1, do not have to be the same as the weights coming into another unit, say Y_2.

- The weights for an M-P neuron are set, together with the threshold for the neuron's activation function, so that the neuron will perform a simple logic function.
- Using these simple neurons as building blocks, we can model any function or phenomenon that can be represented as a logic function.

Example 1. *Implementation of the AND Function Using a McCulloch–Pitts Neuron. The binary forms of the AND function is defined below.*

$$y = x_1 \wedge x_2$$

x_1	x_2	y
0	*0*	*0*
0	*1*	*0*
1	*0*	*0*
1	*1*	*1*

To implement the logical AND function using a McCulloch–Pitts neuron, consider two binary inputs x_1 and x_2, where each input can take the value 0 or 1. Let both inputs be excitatory and assign each of them an equal weight of 1. Choose the threshold of the neuron as

$$\theta = 2$$

The total input to the neuron is therefore

$$y_{in} = x_1 + x_2$$

According to the McCulloch–Pitts activation rule, the neuron produces an output of 1 only if the total input is greater than or equal to the threshold; otherwise, the output is 0. Thus,

$$y = \begin{cases} 1, & \text{if } x_1 + x_2 \geq 2 \\ 0, & \text{if } x_1 + x_2 < 2 \end{cases}$$

Now consider all possible input combinations:

- *If $x_1 = 0$ and $x_2 = 0$, then $y_{in} = 0$, so the output is 0.*
- *If $x_1 = 0$ and $x_2 = 1$, then $y_{in} = 1$, so the output is 0.*
- *If $x_1 = 1$ and $x_2 = 0$, then $y_{in} = 1$, so the output is 0.*
- *If $x_1 = 1$ and $x_2 = 1$, then $y_{in} = 2$, so the output is 1.*

Hence, the output of the neuron is 1 only when both inputs are 1. This exactly matches the truth table of the logical AND function. Therefore, a McCulloch–Pitts neuron with two excitatory inputs of weight 1 and threshold 2 successfully implements the AND gate.

Example 2. *Implementation of the OR Function Using a McCulloch–Pitts Neuron.*

To implement the logical OR function using a McCulloch–Pitts neuron, consider two binary inputs x_1 and x_2, where each input can take the value 0 or 1. Let both inputs be excitatory and assign each of them an equal weight of 1. Choose the threshold of the neuron as

$$\theta = 1$$

Note that the OR function fires if at least one input is active. Therefore, the minimum number of required excitatory inputs is $k = 1$. Using the threshold condition

$$(1)(1) \geq \theta > (0)(1)$$

which simplifies to

$$1 \geq \theta > 0$$

Hence, choosing

$$\theta = 1$$

gives the appropriate threshold for implementing the OR function.

The total input to the neuron is therefore

$$y_{in} = x_1 + x_2$$

According to the McCulloch–Pitts activation rule, the neuron produces an output of 1 if the total input is greater than or equal to the threshold; otherwise, the output is 0. Thus,

$$y = \begin{cases} 1, & \text{if } x_1 + x_2 \geq 1 \\ 0, & \text{if } x_1 + x_2 < 1 \end{cases}$$

Now consider all possible input combinations:

- *If* $x_1 = 0$ *and* $x_2 = 0$*, then* $y_{in} = 0$*, so the output is 0.*
- *If* $x_1 = 0$ *and* $x_2 = 1$*, then* $y_{in} = 1$*, so the output is 1.*
- *If* $x_1 = 1$ *and* $x_2 = 0$*, then* $y_{in} = 1$*, so the output is 1.*
- *If* $x_1 = 1$ *and* $x_2 = 1$*, then* $y_{in} = 2$*, so the output is 1.*

Hence, the output of the neuron is 1 whenever at least one of the inputs is 1. This exactly matches the truth table of the logical OR function. Therefore, a McCulloch–Pitts neuron with two excitatory inputs of weight 1 and threshold 1 successfully implements the OR gate.

$$y = x_1 \vee x_2$$

x_1	x_2	y
0	*0*	*0*
0	*1*	*1*
1	*0*	*1*
1	*1*	*1*

Example 3. *Implementation of the ANDNOT Function Using a McCulloch–Pitts Neuron.*

To implement the logical ANDNOT function using a McCulloch–Pitts neuron, consider two binary inputs x_1 *and* x_2*, where each input can take the value 0 or 1. The function ANDNOT means that the output should be 1 only when* $x_1 = 1$ *and* $x_2 = 0$*. In other words,*

$$y = x_1 \wedge \overline{x_2}$$

To realize this using a McCulloch–Pitts neuron, let x_1 *be an excitatory input with weight 1 and let* x_2 *be an inhibitory input. Choose the threshold of the neuron as*

$$\theta = 1$$

The neuron fires only if it receives sufficient excitatory input and no inhibitory input is active. Since inhibition is absolute in the McCulloch–Pitts model, any active inhibitory input will prevent the neuron from firing.

Now consider all possible input combinations:

- *If* $x_1 = 0$ *and* $x_2 = 0$*, then there is no excitatory input, so the neuron does not fire and the output is 0.*
- *If* $x_1 = 0$ *and* $x_2 = 1$*, then the inhibitory input is active, so the neuron does not fire and the output is 0.*

- *If $x_1 = 1$ and $x_2 = 0$, then the excitatory input is active and no inhibitory input is present, so the neuron fires and the output is 1.*
- *If $x_1 = 1$ and $x_2 = 1$, then although the excitatory input is active, the inhibitory input is also active, and because inhibition is absolute, the neuron does not fire. Hence the output is 0.*

Thus, the output is 1 only for the input combination $(1, 0)$. This exactly matches the truth table of the logical ANDNOT function.

x_1	x_2	y
0	*0*	*0*
0	*1*	*0*
1	*0*	*1*
1	*1*	*0*

Therefore, a McCulloch–Pitts neuron with one excitatory input x_1, one inhibitory input x_2, and threshold 1 successfully implements the ANDNOT gate.

1.4 Merits and Limitations of McCulloch–Pitts Model

Merits

- It was the **first formal neuron model**.
- It showed that neurons can perform **logical computation**.
- It laid the foundation for later neural network models.

Limitations

- Inputs and outputs are only **binary**.
- Weights are **fixed**; there is no learning.
- It is not suitable for noisy or continuous-valued real-world data.
- It is mainly a **logic model**, not a trainable classifier.

2 Perceptron Neuron Model

The McCulloch–Pitts model is useful for logic design, but it does not learn from examples. Its weights and threshold are fixed in advance, which makes it suitable for implementing simple logical functions but not for solving practical classification problems. To make artificial neurons more useful in pattern recognition and decision-making tasks, the next step was to introduce adjustable weights, a trainable decision boundary, and a learning rule. This development led to the **perceptron**.

The perceptron neuron model is an early artificial neuron model developed to perform binary classification. It extends the idea of the McCulloch–Pitts neuron by introducing adjustable weights, so that the neuron can learn from training examples instead of relying

only on fixed logical rules. A perceptron receives several input values, multiplies each input by a corresponding weight, adds a bias term, and then applies a threshold activation function to produce an output.

Thus, the perceptron is an early trainable neural model. Like the McCulloch–Pitts neuron, it is based on threshold logic. However, unlike the McCulloch–Pitts model, it can modify its weights during learning. Because of this property, the perceptron can act as a **linear classifier**, capable of separating input patterns into two classes whenever the data are linearly separable.

The perceptron represents an important step in the evolution of neural networks because it combines the logical neuron concept with a mechanism for learning from data. This makes it a fundamental model in the study of artificial neural networks and machine learning.

2.1 Architecture of a Perceptron

Suppose there are n input signals:

$$x_1, x_2, \ldots, x_n$$

with corresponding weights:

$$w_1, w_2, \ldots, w_n$$

and a bias b.
The net input to the perceptron is:

$$y_{in} = \sum_{i=1}^{n} w_i x_i + b \tag{4}$$

The activation function is a threshold function:

$$f(y_{in}) = \begin{cases} 1, & \text{if } y_{in} \geq 0 \\ 0, & \text{if } y_{in} < 0 \end{cases} \tag{5}$$

The perceptron decides whether an input belongs to class 1 or class 0 based on the sign of the weighted sum.
Thus:

- the M-P neuron is a fixed logical threshold device,
- the perceptron is a threshold device with **learnable weights**.

Example 4. *AND Gate Using Perceptron.*

To implement the logical AND function using a perceptron, consider two binary inputs x_1 and x_2, where each input can take the value 0 or 1. Let the perceptron have weights

$$w_1 = 1, \qquad w_2 = 1$$

and bias

$$b = -1.5$$

The net input to the perceptron is

$$y_{in} = w_1 x_1 + w_2 x_2 + b$$

Substituting the given values, we get

$$y_{in} = x_1 + x_2 - 1.5$$

The perceptron activation rule is

$$y = \begin{cases} 1, & \text{if } y_{in} \geq 0 \\ 0, & \text{if } y_{in} < 0 \end{cases}$$

Now consider all possible input combinations:

- *If* $x_1 = 0$ *and* $x_2 = 0$*, then*

  $$y_{in} = 0 + 0 - 1.5 = -1.5$$

  Since $y_{in} < 0$*, the output is* $y = 0$*.*

- *If* $x_1 = 0$ *and* $x_2 = 1$*, then*

  $$y_{in} = 0 + 1 - 1.5 = -0.5$$

  Since $y_{in} < 0$*, the output is* $y = 0$*.*

- *If* $x_1 = 1$ *and* $x_2 = 0$*, then*

  $$y_{in} = 1 + 0 - 1.5 = -0.5$$

  Since $y_{in} < 0$*, the output is* $y = 0$*.*

- *If* $x_1 = 1$ *and* $x_2 = 1$*, then*

  $$y_{in} = 1 + 1 - 1.5 = 0.5$$

  Since $y_{in} \geq 0$*, the output is* $y = 1$*.*

Thus, the output table is

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

This is exactly the truth table of the logical AND function. Therefore, the perceptron with weights $w_1 = 1$*,* $w_2 = 1$*, and bias* $b = -1.5$ *successfully implements the AND gate.*

Example 5. *To implement the logical OR function using a perceptron, consider two binary inputs x_1 and x_2, where each input can take the value 0 or 1. Let the perceptron have weights*

$$w_1 = 1, \qquad w_2 = 1$$

and bias

$$b = -0.5$$

The net input to the perceptron is

$$y_{in} = w_1 x_1 + w_2 x_2 + b$$

Substituting the given values, we get

$$y_{in} = x_1 + x_2 - 0.5$$

The perceptron activation rule is

$$y = \begin{cases} 1, & \text{if } y_{in} \geq 0 \\ 0, & \text{if } y_{in} < 0 \end{cases}$$

Now consider all possible input combinations:

- *If $x_1 = 0$ and $x_2 = 0$, then*

  $$y_{in} = 0 + 0 - 0.5 = -0.5$$

  Since $y_{in} < 0$, the output is $y = 0$.

- *If $x_1 = 0$ and $x_2 = 1$, then*

  $$y_{in} = 0 + 1 - 0.5 = 0.5$$

  Since $y_{in} \geq 0$, the output is $y = 1$.

- *If $x_1 = 1$ and $x_2 = 0$, then*

  $$y_{in} = 1 + 0 - 0.5 = 0.5$$

  Since $y_{in} \geq 0$, the output is $y = 1$.

- *If $x_1 = 1$ and $x_2 = 1$, then*

  $$y_{in} = 1 + 1 - 0.5 = 1.5$$

  Since $y_{in} \geq 0$, the output is $y = 1$.

Thus, the output table is

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

This is exactly the truth table of the logical OR function. Therefore, the perceptron with weights $w_1 = 1$, $w_2 = 1$, and bias $b = -0.5$ successfully implements the OR gate.

Example 6. *To implement the logical NOT function using a perceptron, consider a single binary input x, where $x \in \{0, 1\}$. Let the perceptron have weight*

$$w = -1$$

and bias

$$b = 0.5$$

The net input to the perceptron is

$$y_{in} = wx + b$$

Substituting the given values, we get

$$y_{in} = -x + 0.5$$

The perceptron activation rule is

$$y = \begin{cases} 1, & \text{if } y_{in} \geq 0 \\ 0, & \text{if } y_{in} < 0 \end{cases}$$

Now consider all possible input values:

- *If $x = 0$, then*

$$y_{in} = -(0) + 0.5 = 0.5$$

  Since $y_{in} \geq 0$, the output is $y = 1$.

- *If $x = 1$, then*

$$y_{in} = -(1) + 0.5 = -0.5$$

  Since $y_{in} < 0$, the output is $y = 0$.

Thus, the output table is

x	y
0	1
1	0

This is exactly the truth table of the logical NOT function. Therefore, the perceptron with weight $w = -1$ and bias $b = 0.5$ successfully implements the NOT gate.

2.2 Perceptron Learning Rule

The most important improvement in perceptron logic is that weights can be updated automatically.
For each training pattern:

- Compute the actual output y,

- Compare it with desired target t: If the output is correct, then $t - y = 0$, so no update is needed. Otherwise, update the weights.

  The perceptron weight update rule is:

$$w_i^{new} = w_i^{old} + \eta(t - y)x_i \tag{6}$$

  and the bias update is:

$$b^{new} = b^{old} + \eta(t - y) \tag{7}$$

  where:

  - η is the learning rate,
  - t is the target output,
  - y is the actual output.

Note: A perceptron can solve only those problems for which the classes can be separated by a straight line (in 2D), a plane (in 3D), or a hyperplane (in higher dimensions). Such problems are called **linearly separable**.

- AND is linearly separable.
- OR is linearly separable.
- NAND is linearly separable.

Note: The XOR function is defined by:

x_1	x_2	XOR output
0	0	0
0	1	1
1	0	1
1	1	0

A single perceptron cannot solve XOR because the data are **not linearly separable**. This is a major limitation of the perceptron and one of the reasons multilayer neural networks were developed.

2.3 Decision Boundary

A decision boundary is the line, plane, or hyperplane that separates different classes in a classification problem. In a perceptron, it is obtained by setting the net input equal to zero:

$$w_1x_1 + w_2x_2 + b = 0$$

This boundary divides the input space into two regions, each corresponding to a different output class.

Example 7. *Training the logical AND function for bipolar inputs and targets using perceptron learning rule.*

Consider the logical AND function with bipolar inputs and targets, where

$$x_1, x_2 \in \{-1, +1\}, \qquad t \in \{-1, +1\}$$

The truth table for the bipolar AND function is

x_1	x_2	t
-1	-1	-1
-1	$+1$	-1
$+1$	-1	-1
$+1$	$+1$	$+1$

The perceptron net input is given by

$$y_{in} = w_1 x_1 + w_2 x_2 + b$$

and the output rule for bipolar targets is

$$y = \begin{cases} +1, & \text{if } y_{in} > 0 \\ -1, & \text{if } y_{in} \leq 0 \end{cases}$$

The perceptron learning rule is applied only when a pattern is misclassified:

$$w_i^{new} = w_i^{old} + \eta t x_i$$

$$b^{new} = b^{old} + \eta t$$

Assume the following initial conditions:

$$\eta = 1, \qquad w_1 = 0, \qquad w_2 = 0, \qquad b = 0$$

Epoch 1

Pattern 1: $(x_1, x_2) = (-1, -1),\ t = -1$

$$y_{in} = 0(-1) + 0(-1) + 0 = 0$$

Since $y_{in} \leq 0$, the output is

$$y = -1$$

This is correct, so no update is made.

Pattern 2: $(x_1, x_2) = (-1, +1),\ t = -1$

$$y_{in} = 0(-1) + 0(+1) + 0 = 0$$

Thus,

$$y = -1$$

This is correct, so no update is made.

Pattern 3: $(x_1, x_2) = (+1, -1),\ t = -1$

$$y_{in} = 0(+1) + 0(-1) + 0 = 0$$

Thus,

$$y = -1$$

This is correct, so no update is made.

Pattern 4: $(x_1, x_2) = (+1, +1),\ t = +1$

$$y_{in} = 0(+1) + 0(+1) + 0 = 0$$

Thus,

$$y = -1$$

This is incorrect, so update the weights and bias:

$$w_1^{new} = 0 + 1(1)(1) = 1$$

$$w_2^{new} = 0 + 1(1)(1) = 1$$

$$b^{new} = 0 + 1(1) = 1$$

So after Epoch 1,

$$w_1 = 1, \qquad w_2 = 1, \qquad b = 1$$

Epoch 2

Pattern 1: $(x_1, x_2) = (-1, -1),\ t = -1$

$$y_{in} = 1(-1) + 1(-1) + 1 = -1$$

Thus,

$$y = -1$$

This is correct, so no update is made.

Pattern 2: $(x_1, x_2) = (-1, +1),\ t = -1$

$$y_{in} = 1(-1) + 1(+1) + 1 = 1$$

Thus,

$$y = +1$$

This is incorrect, so update:

$$w_1^{new} = 1 + 1(-1)(-1) = 2$$

$$w_2^{new} = 1 + 1(-1)(+1) = 0$$

$$b^{new} = 1 + 1(-1) = 0$$

So now,

$$w_1 = 2, \qquad w_2 = 0, \qquad b = 0$$

Pattern 3: $(x_1, x_2) = (+1, -1),\ t = -1$

$$y_{in} = 2(+1) + 0(-1) + 0 = 2$$

Thus,

$$y = +1$$

This is incorrect, so update:

$$w_1^{new} = 2 + 1(-1)(1) = 1$$

$$w_2^{new} = 0 + 1(-1)(-1) = 1$$

$$b^{new} = 0 + 1(-1) = -1$$

So now,

$$w_1 = 1, \qquad w_2 = 1, \qquad b = -1$$

Pattern 4: $(x_1, x_2) = (+1, +1),\ t = +1$

$$y_{in} = 1(+1) + 1(+1) - 1 = 1$$

Thus,

$$y = +1$$

This is correct, so no update is made.

Epoch 3: Convergence Check

Using

$$w_1 = 1, \qquad w_2 = 1, \qquad b = -1$$

Pattern 1: $(-1, -1)$

$$y_{in} = 1(-1) + 1(-1) - 1 = -3 \Rightarrow y = -1$$

Pattern 2: $(-1, +1)$

$$y_{in} = 1(-1) + 1(+1) - 1 = -1 \Rightarrow y = -1$$

Pattern 3: $(+1, -1)$

$$y_{in} = 1(+1) + 1(-1) - 1 = -1 \Rightarrow y = -1$$

Pattern 4: $(+1, +1)$

$$y_{in} = 1(+1) + 1(+1) - 1 = 1 \Rightarrow y = +1$$

All patterns are classified correctly. Therefore, the perceptron has converged.

Final Trained Perceptron

The final trained weights and bias are

$$\boxed{w_1 = 1, \qquad w_2 = 1, \qquad b = -1}$$

Thus, the trained perceptron is

$$y_{in} = x_1 + x_2 - 1$$

with output rule

$$y = \begin{cases} +1, & \text{if } x_1 + x_2 - 1 > 0 \\ -1, & \text{if } x_1 + x_2 - 1 \leq 0 \end{cases}$$

Decision Boundary

The decision boundary is obtained by setting

$$y_{in} = 0$$

Hence,

$$x_1 + x_2 - 1 = 0$$

or equivalently,

$$\boxed{x_1 + x_2 = 1}$$

Therefore, the perceptron successfully learns the bipolar AND function.

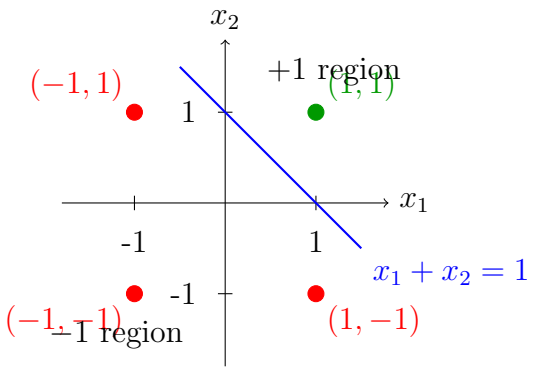


Figure 1: Decision boundary of the perceptron for the bipolar AND function.

3 Usefulness in Modern Neural Networks

Although the McCulloch–Pitts neuron and the simple perceptron are not used directly in most modern deep learning systems, they remain very important because:

1. they provide the **basic concept of an artificial neuron**,
2. they explain how **threshold logic** works,

3. they introduce the ideas of **weights, bias, activation, and decision boundary**,
4. they form the conceptual basis of **multilayer perceptrons** and modern neural networks.

Thus, their importance today is mainly **foundational and educational**.

4 Modern-World Relevance and Examples

Although the McCulloch–Pitts neuron and perceptron are early and simplified models, their ideas remain highly relevant in the modern world.

Why These Models Still Matter Today

The McCulloch–Pitts neuron introduced the idea that a neuron can be viewed as a **binary decision-making unit**. This is still useful today in understanding:

- digital logic circuits,
- rule-based decision systems,
- threshold-based alarms and control mechanisms.

The perceptron introduced the idea of a **weighted input sum followed by a decision rule**. This basic idea still survives in:

- binary classifiers,
- logistic regression,
- multilayer perceptrons,
- deep neural networks used in image, speech, and text processing.

Thus, while modern neural networks are much more complex, the basic building block still follows the same pattern:

$$\text{input} \to \text{weighted sum} \to \text{activation} \to \text{output}$$

Example 8. ***Email Spam Detection***
Suppose we want to classify whether an email is spam.
Let:

- $x_1 = 1$ *if the email contains suspicious words, else 0,*
- $x_2 = 1$ *if the email contains many links, else 0,*
- $x_3 = 1$ *if sender is unknown, else 0.*

Assume:

$$w_1 = 0.8, \quad w_2 = 0.6, \quad w_3 = 0.7, \quad b = -1.0$$

Then:

$$y_{in} = 0.8x_1 + 0.6x_2 + 0.7x_3 - 1.0$$

If an email has:

$$x_1 = 1, \quad x_2 = 1, \quad x_3 = 1$$

then:

$$y_{in} = 0.8 + 0.6 + 0.7 - 1.0 = 1.1$$

Since $y_{in} > 0$, the perceptron classifies the email as spam.

This example shows how perceptron logic can be applied to a real-world classification task.

Example 9. *Consider a medical alarm system with three binary inputs:*

- $x_1 =$ *high temperature,*
- $x_2 =$ *low oxygen,*
- $x_3 =$ *abnormal heart rate.*

Assume each input has excitatory weight $w = 1$ and threshold $\theta = 2$.

Then the alarm neuron fires only if at least two danger signals are active:

$$y = \begin{cases} 1, & \text{if } x_1 + x_2 + x_3 \geq 2 \\ 0, & \text{otherwise} \end{cases}$$

This is a practical modern example of threshold logic similar to the McCulloch–Pitts neuron.

ADALINE and MADALINE Neural Networks

1 Introduction

The development of neural networks did not move directly from simple logic-based neuron models to modern deep learning systems. An important intermediate stage in this evolution was the introduction of trainable linear units and early multilayer adaptive networks. Two historically significant models in this progression are **ADALINE** and **MADALINE**.

The **Adaptive Linear Neuron (ADALINE)** was introduced in 1959 by Bernard Widrow and Ted Hoff. It represented an important improvement over the perceptron because it trained its weights using a continuous linear response rather than relying only on hard-threshold output decisions. ADALINE learns by minimizing the **Mean Squared Error (MSE)** using the **delta rule**, also known as the **Least Mean Squares (LMS)** or **Widrow–Hoff learning rule**.

A **MADALINE** (*Many ADALINE*) network is formed by combining multiple ADALINE units into a multilayer structure. This extension increased the representational capability of the model and made MADALINE one of the earliest important multilayer neural network architectures. However, the presence of hidden units also made the training process more complex.

Earlier neural models such as the McCulloch–Pitts neuron were mainly designed for logical computation, while the perceptron introduced trainable weights for binary classification. However, perceptron learning depends on hard-threshold output error and is guaranteed to converge only when the training data is linearly separable.

ADALINE provided a more mathematically grounded approach to learning. Although its final output may still be thresholded for classification, the training process is based on the linear output before thresholding. This idea is the key conceptual innovation behind ADALINE. Later, by combining several ADALINE units, researchers developed MADALINE, which became an important milestone in the history of multilayer neural computation.

2 ADALINE Neural Network

2.1 Basic Idea

- **Inputs** $(x_1, x_2, \ldots, x_n)$: the feature values fed into the neuron.
- **Weights** $(w_1, w_2, \ldots, w_n)$: adjustable parameters that determine how strongly each input influences the neuron.

- **Bias** (b): an offset term, often treated as a weight connected to a constant input $x_0 = 1$.
- **Net input function**: computes the weighted sum

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- **Activation function**: in ADALINE, the learning is based on a linear activation, so the output before thresholding is essentially

$$y = f(z) = z$$

- **Output function / quantizer**: after the linear response is computed, a threshold or step function can be applied to produce the final class label, such as -1 or $+1$.

This is one of the key differences from the perceptron: **ADALINE updates its weights using the linear output before thresholding, not the final binary decision.**

2.2 Architecture

Let the input vector be

$$\mathbf{x} = (x_1, x_2, \ldots, x_n)$$

and the corresponding weight vector be

$$\mathbf{w} = (w_1, w_2, \ldots, w_n).$$

Then the net input is

$$z = \sum_{i=1}^{n} w_i x_i + b$$

where b is the bias term. If the bias is incorporated into the input by taking $x_0 = 1$ and $w_0 = b$, the equation can be written as

$$z = \sum_{i=0}^{n} w_i x_i.$$

The activation in ADALINE is **linear**, so the output used during training is

$$y = f(z) = z.$$

For classification tasks, a threshold or sign function may be applied after this stage to generate the final binary class label. The important architectural idea is that learning is based on the *linear output before thresholding.*

2.3 Delta Rule / LMS Learning

The learning rule used in **ADALINE** is the **Delta Rule**, also known as the **LMS (Least Mean Squares) rule** or the **Widrow–Hoff learning rule**. This rule adjusts the weights so that the network minimizes the difference between the desired target output and the actual linear output of the neuron.

Basic Idea

Unlike the perceptron, which updates its weights using the thresholded class output, ADALINE performs learning using the **linear output before thresholding**. For an input vector

$$\mathbf{x} = (x_1, x_2, \ldots, x_n)$$

with corresponding weights

$$\mathbf{w} = (w_1, w_2, \ldots, w_n),$$

the net input is

$$z = \sum_{i=1}^{n} w_i x_i + b$$

and since ADALINE uses a linear activation function,

$$y = f(z) = z.$$

If the desired target output is t, then the error is defined as

$$e = t - y.$$

Mean Squared Error (MSE)

The Delta Rule is derived by minimizing the **Mean Squared Error (MSE)**. For a single training example, the error function is

$$E = \frac{1}{2}(t - y)^2.$$

For a set of training examples, the total error is

$$E = \frac{1}{2} \sum_{p} (t^{(p)} - y^{(p)})^2$$

where p denotes the training pattern.

The factor $\frac{1}{2}$ is included for mathematical convenience when taking derivatives.

Weight Update Rule

The aim is to update the weights in the direction that reduces the error. Using gradient descent, the change in each weight is given by

$$\Delta w_i = \eta(t - y)x_i$$

and the bias is updated as

$$\Delta b = \eta(t - y)$$

where η is the learning rate.

Thus, the new weights become

$$w_i^{\text{new}} = w_i^{\text{old}} + \eta(t - y)x_i$$

and

$$b^{\text{new}} = b^{\text{old}} + \eta(t - y).$$

Learning Steps

The LMS learning procedure can be summarized as follows:

1. Initialize the weights and bias to small random values or zeros.
2. Present an input vector $\mathbf{x}$ to the network.
3. Compute the net input
$$z = \sum_{i=1}^{n} w_i x_i + b.$$
4. Compute the linear output
$$y = z.$$
5. Compute the error
$$e = t - y.$$
6. Update the weights and bias using
$$w_i \leftarrow w_i + \eta e x_i$$
$$b \leftarrow b + \eta e.$$
7. Repeat the process for all training examples until the total error becomes sufficiently small.

Selection of Learning Rate η

In gradient-descent learning algorithms such as the **Delta Rule** or **Least Mean Squares (LMS)** algorithm, the learning rate η (sometimes denoted by α or μ) controls the step size of each weight update. Choosing an appropriate value of η is crucial for successful learning.

- If α is **too large**, the algorithm may overshoot the minimum of the error surface, oscillate, and even diverge.
- If α is **too small**, the algorithm converges very slowly, leading to high computational cost.

Thus, learning-rate selection is a trade-off between **stability** and **speed of convergence**.

Theoretical Stability Bound (Hecht-Nielsen (1990))

For ADALINE and LMS learning, the error surface is quadratic, and its geometry is determined by the **input correlation matrix**

$$R = \frac{1}{P} \sum_{p=1}^{P} x^{(p)} \left(x^{(p)}\right)^T$$

where

- P is the number of training samples,
- $x^{(p)}$ is the input vector for the p-th sample,
- $\left(x^{(p)}\right)^T$ is its transpose.

Let $\lambda_{\max}$ denote the largest eigenvalue of R. This eigenvalue corresponds to the direction of greatest curvature of the quadratic error surface.

For the commonly used LMS update equation,

$$w(k+1) = w(k) + \alpha\, e(k)\, x(k),$$

a standard mean-stability condition is

$$0 < \alpha < \frac{2}{\lambda_{\max}}.$$

Remark: In some texts, the update equation is written with a factor of 2 inside the gradient term. Under that convention, the bound is often written as

$$0 < \alpha < \frac{1}{\lambda_{\max}}.$$

Therefore, the exact constant depends on the chosen notation, but the main idea remains the same: **the learning rate must be inversely related to the largest eigenvalue of the correlation matrix.**

Practical Engineering View Although the eigenvalue-based condition is mathematically precise, it is often inconvenient in practice because computing the matrix R and its eigenvalues can be expensive for large problems.

For this reason, engineers often use empirical heuristics when selecting η.

- **Rule 1: Start Small**

  A common practical strategy is to begin with a small value such as

  $$\eta = 0.1$$

  or smaller, and then reduce it if the training error becomes unstable or starts increasing.

  Rule 2: Scale with the Number of Inputs

  For a single ADALINE neuron, a widely used practical guideline is

  $$0.1 \leq n\eta \leq 1,$$

  where n is the number of input units.

  Rearranging gives

  $$\frac{0.1}{n} \leq \eta \leq \frac{1}{n}.$$

  This heuristic reflects the fact that the total input energy tends to grow with the number of inputs. Therefore, as n increases, the learning rate should generally decrease.

Example 1. *Suppose an ADALINE network is designed to process an image patch with*

$$n = 100$$

input pixels.

Using the practical heuristic

$$\frac{0.1}{n} \leq \eta \leq \frac{1}{n},$$

we obtain

$$\frac{0.1}{100} \leq \eta \leq \frac{1}{100}.$$

Hence,

$$0.001 \leq \eta \leq 0.01.$$

So a reasonable practical choice would be a value inside this interval, for example

$$\eta = 0.005.$$

Note: In perceptron learning, the update is based on the final thresholded output. In ADALINE, the update is based on the continuous linear output before thresholding. Therefore, ADALINE can measure *how far* the output is from the target, not just whether the classification is correct or wrong.

2.4 Strengths and Limitations of ADALINE

2.4.1 Strengths

- Simple and mathematically elegant model.
- Adjustable weights and bias.
- Training directly minimizes a squared-error objective.
- Historically important in adaptive systems and signal processing.

2.4.2 Limitations

- It is still a **single-layer linear model**.
- It cannot directly solve nonlinearly separable problems such as XOR.
- Although the output is thresholded, the decision boundary remains linear.

3 MADALINE Neural Network

3.1 Basic Idea

A typical MADALINE network consists of three layers:

- **Input Layer:** Receives the input feature signals.
- **Hidden Layer:** Contains multiple ADALINE units. Each unit computes a weighted sum of its inputs and produces a response that is commonly passed through a hard-limiting sign function, yielding outputs such as $+1$ or -1.
- **Output Layer:** Combines the outputs of the hidden units to produce the final network response.

3.2 Simple MADALINE Structure

A simple MADALINE may contain:

- two hidden ADALINE units Z_1 and Z_2,
- one output ADALINE unit Y.

Each ADALINE simply applies a threshold function to the unit's net input. The output Y is therefore a non-linear function of the input vector (X_1, X_2) because of the presence of the hidden units.

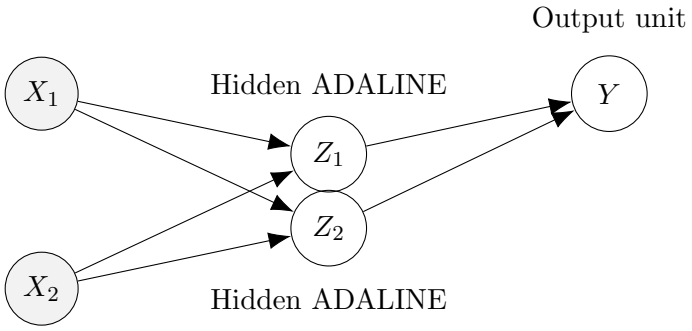


Figure 1: A simple MADALINE network with two hidden ADALINE units and one output unit.

3.3 Need for Hidden Units

A single ADALINE can only produce a linear decision boundary. By introducing hidden ADALINE units, MADALINE can combine several linear decisions into a more powerful non-linear overall decision function.
Thus:

- hidden units increase computational power,
- multilayer structure enables more complex mappings,
- but training hidden-layer weights becomes more challenging.

This is the central motivation for studying MADALINE as an early multilayer neural architecture.

3.4 Training Algorithms for MADALINE

There are two classical training algorithms for a MADALINE with one hidden layer:

- **MR-I Algorithm:** The original MADALINE training algorithm due to Widrow and Hoff (1960). It changes the weights onto the hidden ADALINEs only. The output-unit weights are fixed, and the algorithm assumes that the output unit is an OR unit.
- **MR-II Algorithm:** A later algorithm that adjusts all weights in the network and does not assume that the output unit is an OR unit.

3.4.1 MR-I Algorithm

Activation Function For units Z_1, Z_2, and Y, use the bipolar threshold function

$$f(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ -1, & \text{if } x < 0 \end{cases} \tag{1}$$

Initialization The unit Y performs the logic function OR on the signals it receives from Z_1 and Z_2. The weights into Y are fixed as

$$v_1 = \frac{1}{2}, \qquad v_2 = \frac{1}{2}, \qquad b_y = \frac{1}{2}$$

Small random values are usually used for the hidden-layer ADALINE weights and biases. Set a small learning rate η.

Algorithm Steps

Step 0: Initialize weights.
Step 1: While stopping condition is false, do Steps 2–8.
Step 2: For each bipolar training pair $\mathbf{s} : t$, do Steps 3–7.
Step 3: Set activation for input units

$$x_i = s_i \qquad \forall i = 1, 2, \ldots, n$$

Step 4: Compute net input to each hidden ADALINE unit

$$z_{in1} = b_1 + x_1 w_{11} + x_2 w_{21}$$

$$z_{in2} = b_2 + x_1 w_{12} + x_2 w_{22}$$

Step 5: Determine output of each hidden ADALINE unit

$$z_1 = f(z_{in1}), \qquad z_2 = f(z_{in2})$$

Step 6: Determine output of the network

$$y_{in} = b_y + z_1 v_1 + z_2 v_2$$

$$y = f(y_{in})$$

Step 7: Determine error and update weights

If $y = t$, no weight update is performed.

Otherwise:

- If $t = 1$, then update the weights on the hidden unit whose net input is closest to 0. This is the unit that can most easily change its output.

$$b_j(\text{new}) = b_j(\text{old}) + \alpha\big(1 - z_{inj}\big)$$
$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha\big(1 - z_{inj}\big)x_i$$

- If $t = -1$, then update the weights on all hidden units whose net inputs are positive:

$$b_j(\text{new}) = b_j(\text{old}) + \alpha\big(-1 - z_{inj}\big)$$
$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha\big(-1 - z_{inj}\big)x_i$$

Step 8: Test stopping condition.

If weight changes have stopped (or reached an acceptable level), or if a maximum number of iterations has been performed, then stop; otherwise continue.

NOTE: MR-I modifies only the hidden ADALINE units. The output unit is fixed and behaves like an OR combiner. Therefore, the learning burden is shifted to the hidden layer.

This algorithm is historically important because it is one of the earliest attempts to train a multilayer neural network without using full backpropagation.

3.4.2 MR-II Algorithm

MR-II is a more general training algorithm. Unlike MR-I, it does not assume that the output unit is an OR unit, and it allows all weights in the network to be adjusted.

Algorithm Steps

Step 0: Initialize weights and set a small learning rate α.

Step 1: While stopping condition is false, do Steps 2–8.

Step 2: For each bipolar training pair $\mathbf{s} : t$, do Steps 3–7.

Steps 3–6: Compute the output of the network as in the MR-I algorithm:

$$x_i = s_i \qquad \forall i = 1, 2, \ldots, n$$

compute hidden net inputs, hidden outputs, output net input, and final output.

Step 7: Determine error and update weights if necessary

If $y \neq t$, then do Steps 7a–7b for each hidden unit whose net input is sufficiently close to 0 (for example, between -0.25 and 0.25). Start with the unit whose net input is closest to 0.

Step 7a: Change the unit's output

$$+1 \leftrightarrow -1$$

Step 7b: Recompute the response of the network.

If the error is reduced, then adjust the weights on this unit using the delta rule, taking its newly assigned output value as the target.

Step 8: Test stopping condition.

If weight changes have stopped (or reached an acceptable level), or if a specified maximum number of iterations has been performed, then stop; otherwise continue.

NOTE: MR-II is more flexible than MR-I because it does not lock the output unit into a fixed OR behavior. It uses a trial-flip strategy on hidden-unit outputs and then applies delta-rule adaptation when improvement is found.

This makes MR-II conceptually important as an early attempt to solve the hidden-unit credit assignment problem before backpropagation became standard.

4 ADALINE vs MADALINE

Feature	**ADALINE**	**MADALINE**
Layers	Single-layer	Multi-layer
Basic unit	One adaptive linear neuron	Many ADALINE units
Learning rule	Delta / LMS rule	MR-I or MR-II
Output nature	Linear response + bipolar threshold	Nonlinear overall response due to hidden units
Capability	Linear classification	More powerful than a single linear unit
Training difficulty	Relatively simple	More complicated due to hidden units

3.4.2 Widrow-Hoff Learning Rule

The Widrow-Hoff learning rule is often termed the Delta rule or the Least Mean Square (LMS) rule. And as the last and most descriptive name - Least Mean Square - indicates, the rule aims at minimizing (the mean square of) the difference between the targeted and the actual data or the response.

3.4.2.1 Algorithm

- The LMS algorithm adjusts the weights and biases of the ADALINE so as to minimize this mean square error.

  Definition 3.4.1 *Let the training set be*

  $$\{\boldsymbol{p}_1, \boldsymbol{t}_1\}, \{\boldsymbol{p}_2, \boldsymbol{t}_2\}, \ldots, \{\boldsymbol{p}_Q, \boldsymbol{t}_Q\} \tag{3.4}$$

  where $\boldsymbol{t}_q$ is the target output, corresponding to the input, $\boldsymbol{p}_q$ for $q = 1, 2, \ldots, Q$. Then, the mean square error(MSE) is

  $$MSE = \frac{1}{Q}\sum_{k=1}^{Q}(t_k - a_k)^2 \tag{3.5}$$

- Consider the single-neuron. For simplicity, let's combine all of the parameters we are adjusting, including the bias, into a single vectors as follows.

  $$\mathbf{x} = \begin{bmatrix} {}_1\mathbf{w} \\ b \end{bmatrix} \tag{3.6}$$

  $$\mathbf{z} = \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix} \tag{3.7}$$

- The network output using a linear activation function,

  $$a = {}_1\mathbf{w}\mathbf{p} + b, \tag{3.8}$$

  can be written as

  $$a = \mathbf{x}^T\mathbf{z} \tag{3.9}$$

- The ADALINE network mean square error[1] can be written as

  $$F(\mathbf{x}) = E[(t - a)^2] = E[(t - \mathbf{x}^T\mathbf{z})^2] \tag{3.10}$$

  where $E[\ ]$ denote for the expected value and it is taken over all sets of input/target pairs.

[1] The mean squared error of $\hat{\theta}$ is defined as the expected value (probability-weighted average, over all samples) of the squared errors; that is,

$$\text{MSE} = E[(\hat{\theta}(X) - \theta)^2].$$

- We can expand this expression as follows:

$$F(\mathbf{x}) = E[t^2 - 2t\mathbf{x}^T\mathbf{z} + (\mathbf{x}^T\mathbf{z})(\mathbf{x}^T\mathbf{z})^T] \tag{3.11}$$[2]

$$\begin{aligned} F(\mathbf{x}) &= E[t^2 - 2t\mathbf{x}^T\mathbf{z} + \mathbf{x}^T\mathbf{z}\mathbf{z}^T\mathbf{x}] \\ &= E[t^2] - 2\mathbf{x}^T E[t\mathbf{z}] + \mathbf{x}^T E[\mathbf{z}\mathbf{z}^T]\mathbf{x} \end{aligned} \tag{3.12}$$

  This can be written in the following form:

$$F(\mathbf{x}) = c - 2\mathbf{x}^T\mathbf{h} + \mathbf{x}^T\mathbf{R}\mathbf{x} \tag{3.13}$$

  where $c = E[t^2]$, $\mathbf{h} = E[t\mathbf{z}]$ and $\mathbf{R} = E[\mathbf{z}\mathbf{z}^T]$.

- Here the vector $\mathbf{h}$ gives the cross-correlation between the input vector and its associated target, while $\mathbf{R}$ is the input *correlation matrix*. The diagonal elements of this matrix are equal to the mean square values of the elements of the input vectors.

- The equation (3.13) is of the form quadratic function

$$F(\mathbf{x}) = c + \mathbf{d}^T\mathbf{x} + \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} \tag{3.14}$$

  when $\mathbf{d} = -2\mathbf{h}$ and $\mathbf{A} = 2\mathbf{R}$.

  Definition 3.4.2 (Quadratic Function) *The general form of a quadratic function is*

$$F(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^T\boldsymbol{A}\boldsymbol{x} + \boldsymbol{d}^T\boldsymbol{x} + c \tag{3.15}$$

  where the matrix $\boldsymbol{A}$ is symmetric.

- Thus the mean square error performance index[3] for the ADALINE network is a quadratic function (3.14).

- Few important characteristics of quadratic function are given below;

  – The gradient of $F(\mathbf{x})$ is

$$\nabla F(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{d} \tag{3.15}$$[4]

  and

$$\nabla^2 F(\mathbf{x}) = \mathbf{A}.$$

  That is, $\mathbf{A}$ is a *Hessian matrix*;

$$A = \begin{bmatrix} \frac{\partial^2 F(\mathbf{x})}{\partial x_1^2} & \cdots & \frac{\partial^2 F(\mathbf{x})}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 F(\mathbf{x})}{\partial x_n x_1} & \cdots & \frac{\partial^2 F(\mathbf{x})}{\partial x_n^2} \end{bmatrix}$$

[2] $\mathbf{x}^T\mathbf{z}$ is symmetric. Therefore, $(\mathbf{x}^T\mathbf{z})^2 = (\mathbf{x}^T\mathbf{z})(\mathbf{x}^T\mathbf{z})^T$

[3] Quantitative measure of network performance.

[4]
* $\nabla(\mathbf{x}^T\mathbf{b}) = \nabla(\mathbf{b}^T\mathbf{x}) = \mathbf{b}$ where $\mathbf{b}$ is a constant matrix.
* $\nabla(\mathbf{x}^T\mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^T)\mathbf{x}$
* $\nabla(\mathbf{x}^T\mathbf{A}\mathbf{x}) = 2\mathbf{A}\mathbf{x}$ if $\mathbf{A}$ is symmetric.

(i) If the eigenvalues of the Hessian matrix are all positive, the function will have a single strong minimum.

(ii) If the eigenvalues are all negative, the function will have a single strong maximum.

(iii) If some eigenvalues are positive and other eigenvalues are negative, the function will have a single saddle point.

(iv) If the eigenvalues are all nonnegative, but some eigenvalues are zero, then the function will either have a weak minimum or will have no stationary point.

(v) If the eigenvalues are all nonpositive, but some eigenvalues are zero, then the function will either have a weak maximum or will have no stationary point.

- In the case of the performance index of the ADALINE network, the Hessian matrix $\mathbf{A}$ of the quadratic function is is twice the correlation matrix $\mathbf{R}$.
- It can be shown that all correlation matrices are either positive definite[5] or positive semidefinite[6], which means that they can never have negative eigenvalues.
- Thus, there are two posibilities left:

  (i) If the correlation matrix has only positive eigenvalues, the performance index will have one unique global minimum.

  (ii) If the correlation matrix has some zero eigenvalues, the performance index will either have a weak minimum or no minimum, depending on the vector $\mathbf{d} = -2\mathbf{h}$.

- To locate the stationary point of the performance index set thee gradient of the quadratic function (3.13) equals to zero.

$$\nabla F(\mathbf{x}) = 2\mathbf{R}\mathbf{x} - 2\mathbf{h} \tag{3.16}$$

$$\begin{aligned} 2\mathbf{R}\mathbf{x} - 2\mathbf{h} &= 0 \\ \mathbf{x} &= \mathbf{R}^{-1}\mathbf{h} \end{aligned} \tag{3.17}$$

- If we could calculate the statistical quantities $\mathbf{h}$ and $\mathbf{R}$, we could find the minimum point directly from Eq. (3.17). In general, however, it is not desirable or convenient to calculate $\mathbf{h}$ and $\mathbf{R}$. For this reason we will use the steepest descent algorithm to estimate the optimum.

  Algorithm 3.4.1 (Steepest Descent Algorithm) *Suppose $F(\boldsymbol{x})$, $\boldsymbol{x} \in R^n$, and $F : R^n \to R$. The stepest descent method produces the approximation to the location of the minimum as follows.*

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k - \alpha_k \nabla F(\boldsymbol{x})|_{\boldsymbol{x}=\boldsymbol{x}_k} \tag{3.18}$$

  where α_k is a positive scalar called learning rate which determines the length of the step.

- The key insight of Widrow and Hoff was that they could estimate the mean square error by

$$F(\mathbf{x}) = (t(k) - a(k))^2 = e^2(k) \tag{3.19}$$

  where the expectation of the squared error has been replaced by the squared error at iteration k.

[5] An $n \times n$ symmetric real matrix M is said to be positive definite if $\mathbf{x}^T M\mathbf{x} > 0$ for all non zero $\mathbf{x} \in R^n$.

[6] An $n \times n$ symmetric real matrix M is said to be positive semidefinite if $\mathbf{x}^T M\mathbf{x} \geq 0$ for all $\mathbf{x} \in R^n$.

- Then, at each iteration we have a gradient estimate of the form:

$$\nabla F(\mathbf{x}) = \nabla e^2(k) \tag{3.20}$$

- Note that the first R elements of $\nabla e^2(k)$ are derivatives with respect to the network weights, while the element $R+1$ is the derivative with respect to the bias. Thus we have

$$[\nabla e^2(k)]_j = \frac{\partial e^2(k)}{\partial w_{1,j}} = 2e(k)\frac{\partial e(k)}{\partial w_{1,j}} = 2e(k)(-p_j(k)) = -2e(k)p_j(k) \tag{3.21}$$

  where $p_j(k)$ is the jth element of the input vector at the kth iteration, for $j = 1, 2, \ldots, R$, and

$$[\nabla e^2(k)]_{R+1} = \frac{\partial e^2(k)}{\partial b} = 2e(k)\frac{\partial e(k)}{\partial b} = 2e(k)(-1) = -2e(k) \tag{3.22}$$

- $p_j(k)$ and 1 are the elements of the input vector $\mathbf{z}$ so the gradient of the squared error at iteration k can be written

$$\nabla F(\mathbf{x}) = \nabla e^2(k) = -2e(k)\mathbf{z}(k) \tag{3.23}$$

- substituting $\nabla F(\mathbf{x})$, from Eqn (3.23) in Eqn (3.18) we find

$$\mathbf{x}_{k+1} = \mathbf{x}_k + 2\alpha_k e(k)\mathbf{z}(k) \tag{3.24}$$

3.4.2.2 The Widrow-Hoff Learning Algorithm [LMS Algorithm]

- We may write the Eqn (3.24) in following manner.

$$_1\mathbf{w}(k+1) = {}_1\mathbf{w}(k) + 2\alpha_k e(k)\mathbf{p}(k) \tag{3.25}$$
$$b(k+1) = b(k) + 2\alpha_k e(k) \tag{3.26}$$

  These last two equations make up the least mean square (LMS) algorithm. This is also referred to as the delta rule or the Widrow-Hoff learning algorithm.

- The preceding results can be modified to handle the case where we have multiple neurons as follows:

$$_i\mathbf{w}(k+1) = {}_i\mathbf{w}(k) + 2\alpha_k e_i(k)\mathbf{p}(k) \tag{3.27}$$
$$b_i(k+1) = b_i(k) + 2\alpha_k e_i(k) \tag{3.28}$$

  where $e_i(k)$ is the ith element of the error at iteration k.

- This algorithm can be written conveniently in matrix notation as follows:

$$\mathbf{W}(k+1) = \mathbf{W}(k) + 2\alpha\mathbf{e}(k)\mathbf{p}^T(k) \tag{3.29}$$
$$\mathbf{b}(k+1) = \mathbf{b}(k) + 2\alpha\mathbf{e}(k) \tag{3.30}$$

Learning Rate

- For steepest descent there are two general methods for determining the learning rate, α_k.

  (i) By minimizing the performance index with respect α_k to at each iteration k.

  (ii) By selecting a fixed value (e.g., $\alpha_k = 0.02$), or to use variable, but predetermined, values (e.g., $\alpha_k = 1/k$).

- There are limitations when selecting the learning rate, α_k. It can be seen that the large learning rate makes the convergence faster. However, if we make the learning rate too large the algorithm will become unstable. There is a maximum allowable learning rate it is shown in the following equation.

$$\alpha < \frac{2}{\lambda_{max}} \tag{3.31}$$

  where λ_{max} is the largest eigenvalue of the Hessian matrix.

- **The Convergence of LMS Algorithm:** It can be shown that the LMS algorithm is convergent when $\alpha < \dfrac{1}{\lambda_{max}}$ where λ_{max} is the largest eigenvalue of the correaltion matrix R.

Example 3.1 *Test the ADALINE network and the LMS algorithm for the following vectors and their associated target.*

$$\left\{\boldsymbol{p}_1 = \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix}, t_1 = [-1]\right\} \qquad \left\{\boldsymbol{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}, t_2 = [1]\right\}$$

Important Definitions and Theorems

Theorem 3.4.1 *If A is an $n \times n$ positive definite matrix, then*

(i) *A has an inverse;*

(ii) *$a_{ii} > 0$, for each $i = 1, 2, ..., n$;*

(iii) *For $k \neq j$, $\max_{1 \leq k,j \leq n} |a_{kj}| \leq \max_{1 \leq i \leq n} |a_{ii}|$;*

(iv) *$(a_{ij})^2 < a_{ii}a_{jj}$, for each $i \neq j$.*

Definition 3.4.3 *A **leading principal submatrix** of a matrix A is a matrix of the form*

$$A_k = \begin{pmatrix} a_{11} & a_{12} & \dots & \dots & a_{1k} \\ a_{21} & a_{22} & \dots & \dots & a_{2k} \\ \dots & \dots & \dots & \dots & \dots \\ \dots & \dots & \dots & \dots & \dots \\ a_{k1} & a_{k2} & \dots & \dots & a_{kk} \end{pmatrix}$$

for some $1 \leq k \leq n$.

Theorem 3.4.2 *A symmetric matrix A is positive definite if and only if each of its leading principal submatrices has a positive determinant.*

Theorem 3.4.3 *A symmetric matrix is positive definite iff all its eigenvalues are positive.*

Theorem 3.4.4 *A symmetric matrix is semi positive definite iff all its eigenvalues are non negative.*

More on ANN Concepts and Topologies

In this lesson, we will further explore neural networks by focusing on the main ANN topologies: the Feedforward Neural Network / Multilayer Perceptron (MLP), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs). We will also discuss their structures, characteristics, and typical applications.

1 Multilayer Perceptrons

A **Multilayer Perceptron (MLP)** is a feedforward artificial neural network made up of an input layer, one or more hidden layers, and an output layer. In an MLP, the output of one layer becomes the input to the next layer. Unlike a single-layer perceptron, an MLP can solve nonlinear problems because the hidden layers and activation functions allow it to learn more complex decision boundaries and internal representations.

1.1 General Structure of an MLP

A multilayer perceptron may be represented as a cascade of layers:

$$a^1 = f^1(W^1 p + b^1),$$

$$a^2 = f^2(W^2 a^1 + b^2),$$

$$a^3 = f^3(W^3 a^2 + b^3),$$

and, in general,

$$a^{m+1} = f^{m+1}(W^{m+1} a^m + b^{m+1}).$$

Where:

- p is the input vector,
- W^m is the weight matrix for layer m,
- b^m is the bias vector for layer m,
- f^m is the transfer or activation function of layer m,
- a^m is the output of layer m.

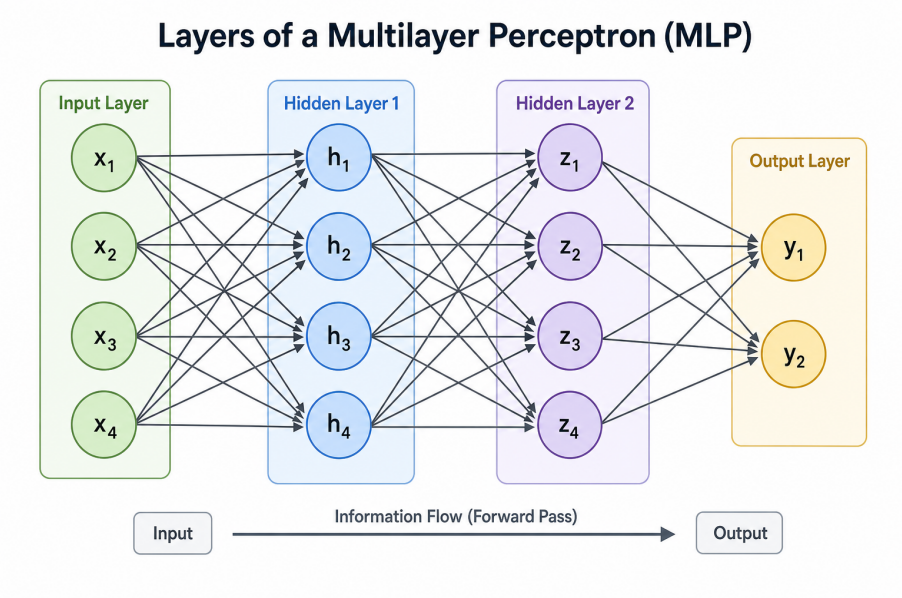


Figure 1: Multilayer Perceptrons

A shorthand notation is often used to describe the architecture:

$$R - S_1 - S_2 - S_3$$

where:

- R is the number of input features,
- S_1 is the number of neurons in the first hidden layer,
- S_2 is the number of neurons in the second hidden layer,
- S_3 is the number of neurons in the output layer.

For example, a network written as

$$2 - 2 - 1$$

has 2 inputs, 2 neurons in one hidden layer, and 1 output neuron.

1.2 Why Multilayer Perceptrons Are Important

A single-layer perceptron can only solve linearly separable problems. This means it cannot correctly classify data when one straight line is not enough to separate the classes. MLPs overcome this limitation by using hidden layers to construct more powerful nonlinear mappings. Hidden units allow the model to learn intermediate features, and activation functions make the network capable of representing complex functions.

Example 1 (XOR Classification Problem)**.** *One of the most famous examples used to illustrate the power of MLPs is the* ***exclusive-or (XOR)*** *problem. The input-target pairs are:*

$$p_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \quad t_1 = 0$$

$$p_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad t_2 = 1$$

$$p_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad t_3 = 1$$

$$p_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad t_4 = 0$$

The XOR problem is not linearly separable. Therefore, a single-layer perceptron cannot solve it. However, a two-layer MLP can solve XOR by constructing internal decision boundaries in the hidden layer and combining them in the output layer. This is one of the classical demonstrations that multilayer networks are more powerful than single-layer networks.

Interpretation of the XOR Solution

A possible 2-2-1 MLP solution works as follows:

- *the first hidden neuron creates one decision boundary,*
- *the second hidden neuron creates another decision boundary,*
- *the output neuron combines the hidden-layer outputs to produce the XOR classification.*

Thus, the hidden layer transforms the original input space into a new representation where the classes become separable.

Example 2 (Function Approximation)**.** *MLPs are also useful as* ***function approximation****. Instead of classifying inputs into categories, the network learns a mapping from input values to continuous outputs.*
Consider a two-layer network with architecture:

$$1 - 2 - 1$$

where:

- *the first layer uses a log-sigmoid activation function,*
- *the second layer uses a linear activation function.*

The equations are:

$$a^1 = logsig(W^1 p + b^1),$$

$$a^2 = purelin(W^2 a^1 + b^2).$$

Suppose the network is used to approximate a nonlinear function such as:

$$g(p) = 1 + \sin\left(\frac{\pi}{4} p\right), \quad -2 \le p \le 2$$

In this case:

- *the hidden layer neurons produce smooth step-like responses,*
- *the output layer combines these responses linearly,*
- *together they approximate the target function.*

This shows that MLPs can model smooth nonlinear relationships, not only classification boundaries. With enough hidden neurons, a two-layer network with sigmoid hidden units and a linear output layer can approximate a very wide class of functions.

Example 3. *Consider the following feedforward neural network with architecture:*

$$2 - 2 - 1$$

The inputs are:

$$x_1 = 1.0, \qquad x_2 = 0.5$$

The hidden-layer weights and biases are:
For hidden neuron 1:

$$w_{11} = 0.2, \qquad w_{21} = 0.7, \qquad b_1 = -0.1$$

For hidden neuron 2:

$$w_{12} = -0.4, \qquad w_{22} = 0.6, \qquad b_2 = 0.2$$

The output-layer weights and bias are:

$$v_1 = 0.9, \qquad v_2 = -0.5, \qquad b_o = 0.1$$

Use:

- *ReLU in the hidden layer,*
- *sigmoid in the output layer.*

Answer the following questions.

a) Compute the net input and output of hidden neuron 1.

b) Compute the net input and output of hidden neuron 2.

c) Compute the net input and final output of the output neuron.

d) If the classification threshold is 0.5*, what class is predicted?*

- *Step 1: Hidden neuron 1*

$$z_1 = (0.2)(1.0) + (0.7)(0.5) - 0.1 = 0.45$$

$$h_1 = ReLU(0.45) = 0.45$$

- *Step 2: Hidden neuron 2*

$$z_2 = (-0.4)(1.0) + (0.6)(0.5) + 0.2 = 0.1$$

$$h_2 = ReLU(0.1) = 0.1$$

- *Step 3: Output neuron*

$$z_o = (0.9)(0.45) + (-0.5)(0.1) + 0.1 = 0.455$$

Apply sigmoid:

$$y = \sigma(0.455) = \frac{1}{1 + e^{-0.455}}$$

$$y \approx 0.612$$

- *Step 4: Classification Since*

$$0.612 > 0.5$$

the predicted class is:

$$\boxed{\textit{Class } 1}$$

1.3 Role of Hidden Neurons

The number of hidden neurons strongly affects the capability of the network:

- too few hidden neurons may cause **underfitting**, where the network cannot capture the complexity of the target function;
- too many hidden neurons may cause **overfitting**, where the network memorizes training points but does not generalize well to unseen data.

In practice, the network should be large enough to learn the problem but not unnecessarily complex. This supports the principle of using the simplest network that performs adequately.

1.4 Activation Functions in MLPs

Each layer in an MLP may use a different activation function. Common choices include:

- **Log-sigmoid:**

$$f(n) = \frac{1}{1 + e^{-n}}$$

- **Tanh:**

$$f(n) = \tanh(n)$$

- **ReLU:**

$$f(n) = \max(0, n)$$

- **Linear:**

$$f(n) = n$$

The hidden layers often use nonlinear activations, while the output layer depends on the task:

- classification may use sigmoid or softmax-like outputs,
- regression often uses a linear output.

1.5 Training Multilayer Perceptrons

Training an MLP means adjusting the weights and biases so that the output of the network becomes close to the target output. This is commonly done by minimizing an error function such as the mean squared error:

$$E = \frac{1}{Q}\sum_{q=1}^{Q}(t_q - a_q)^2$$

where:

- Q is the number of training examples,
- t_q is the target output,
- a_q is the actual network output.

The most common learning algorithm for MLPs is **backpropagation**, which computes gradients efficiently and updates the parameters using gradient descent. Gradient-based learning is a central idea in deep learning.

1.6 Backpropagation: Basic Idea

Backpropagation is one of the most important training algorithms in neural network history. It revived interest in multilayer neural networks by providing a practical way to train feedforward networks with one or more hidden layers. In essence, backpropagation is a gradient descent method used to minimize the total squared error produced by a network.

1.6.1 Main Idea of Backpropagation

Training by backpropagation consists of three stages:

1. **Feedforward:** send the input through the network and compute the output.
2. **Backpropagation of error:** compare the output with the target and propagate the error backward.
3. **Weight update:** adjust weights and biases to reduce the error.

After training, only the feedforward stage is needed for using the network.

1.6.2 Network Architecture

Consider a multilayer feedforward network with:

- an input layer,
- one or more hidden layer,
- an output layer.

Let:

- x_i denote the input signals to input unit i,
- v_{ij} denote weights from input unit X_i to hidden unit Z_j,
- w_{jk} denote weights from hidden unit Z_j to output unit Y_k,
- v_{0j} denote the bias on hidden unit Z_j,
- w_{0k} denote the bias on output unit Y_k.

The hidden layer computes:

$$z_{in,j} = v_{0j} + \sum_{i=1}^{n} x_i v_{ij}, \tag{1}$$

$$z_j = f(z_{in,j}). \tag{2}$$

The output layer computes:

$$y_in, k = w_{0k} + \sum_{j=1}^{p} z_j w_{jk}, \tag{3}$$

$$y_k = f(y_in, k). \tag{4}$$

1.6.3 Activation Functions

For backpropagation, the activation function should be:

- continuous,
- differentiable,
- monotonically non-decreasing,
- easy to differentiate.

The following activation functions were widely used in problems involving backpropagation because their derivatives can be expressed directly in terms of the function output, which simplifies the computations required during training.

- **Binary Sigmoid** A common choice is the binary sigmoid:

  $$f(x) = \frac{1}{1 + e^{-x}},$$

  with derivative

  $$f'(x) = f(x)\big(1 - f(x)\big).$$

  Its range is $(0, 1)$.

- **Bipolar Sigmoid** Another common choice is the bipolar sigmoid:

$$f(x) = \frac{2}{1 + e^{-x}} - 1,$$

with derivative

$$f'(x) = \frac{1}{2}\big(1 + f(x)\big)\big(1 - f(x)\big).$$

Its range is $(-1, 1)$. It is closely related to $\tanh(x)$.

1.6.4 Proof of the Backpropagation Method

This section derives the standard backpropagation update equations for a network with one hidden layer.

- **Network and Error Function**

Consider a feedforward neural network with one hidden layer.

Let

- x_i be the input to input unit X_i,
- v_{ij} be the weight from input unit X_i to hidden unit Z_j,
- w_{jk} be the weight from hidden unit Z_j to output unit Y_k,
- v_{0j} be the bias of hidden unit Z_j,
- w_{0k} be the bias of output unit Y_k.

The net input to hidden unit Z_j is

$$z_{in,j} = v_{0j} + \sum_i x_i v_{ij}, \tag{5}$$

and its output is

$$z_j = f(z_{in,j}). \tag{6}$$

The net input to output unit Y_k is

$$y_{in,k} = w_{0k} + \sum_j z_j w_{jk}, \tag{7}$$

and its output is

$$y_k = f(y_{in,k}). \tag{8}$$

For a single training pattern with target vector $t = (t_1, \ldots, t_m)$, define the squared error by

$$E = \frac{1}{2}\sum_k (t_k - y_k)^2. \tag{9}$$

The goal is to adjust the weights so as to minimize E.

- **Gradient Descent Principle**

  For any weight w, gradient descent updates it by

  $$\Delta w = -\alpha \frac{\partial E}{\partial w}.$$

  Hence, the problem reduces to computing partial derivatives of the error function with respect to each weight and bias.

- **Output-Layer Weight Update**

  Consider the weight w_{jk} from hidden unit Z_j to output unit Y_k. Using the chain rule,

  $$\frac{\partial E}{\partial w_{jk}} = \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial y_{in,k}} \frac{\partial y_{in,k}}{\partial w_{jk}}. \tag{10}$$

  We compute each factor separately:

  $$\frac{\partial E}{\partial y_k} = \frac{\partial}{\partial y_k} \left(\frac{1}{2} \sum_r (t_r - y_r)^2 \right) = -(t_k - y_k),$$

  $$\frac{\partial y_k}{\partial y_{in,k}} = f'(y_{in,k}),$$

  $$\frac{\partial y_{in,k}}{\partial w_{jk}} = z_j.$$

  Hence

  $$\frac{\partial E}{\partial w_{jk}} = -(t_k - y_k) f'(y_{in,k}) z_j. \tag{11}$$

  Define the output error term

  $$\delta_k = (t_k - y_k) f'(y_{in,k}). \tag{12}$$

  Then

  $$\frac{\partial E}{\partial w_{jk}} = -\delta_k z_j. \tag{13}$$

  Applying gradient descent gives

  $$w_{jk}^{\text{new}} = w_{jk}^{\text{old}} - \alpha \frac{\partial E}{\partial w_{jk}} = w_{jk}^{\text{old}} + \alpha \delta_k z_j. \tag{14}$$

  Therefore the correction term is

  $$\Delta w_{jk} = \alpha \delta_k z_j. \tag{15}$$

  This is exactly the standard backpropagation update rule for output-layer weights.

- **Output-Layer Bias Update**

  Since
  $$y_{in,k} = w_{0k} + \sum_j z_j w_{jk}, \tag{16}$$
  we have
  $$\frac{\partial y_{in,k}}{\partial w_{0k}} = 1. \tag{17}$$
  Therefore
  $$\frac{\partial E}{\partial w_{0k}} = \frac{\partial E}{\partial y_k}\frac{\partial y_k}{\partial y_{in,k}}\frac{\partial y_{in,k}}{\partial w_{0k}} = -(t_k - y_k)f'(y_{in,k}) = -\delta_k. \tag{18}$$
  Thus
  $$w_{0k}^{\text{new}} = w_{0k}^{\text{old}} + \alpha\delta_k, \tag{19}$$
  so
  $$\Delta w_{0k} = \alpha\delta_k. \tag{20}$$

- **Hidden-Layer Weight Update**

  Now consider the weight v_{ij} from input unit X_i to hidden unit Z_j. Again by the chain rule,
  $$\frac{\partial E}{\partial v_{ij}} = \frac{\partial E}{\partial z_j}\frac{\partial z_j}{\partial z_{in,j}}\frac{\partial z_{in,j}}{\partial v_{ij}}. \tag{21}$$
  The last two factors are
  $$\frac{\partial z_j}{\partial z_{in,j}} = f'(z_{in,j}), \tag{22}$$
  $$\frac{\partial z_{in,j}}{\partial v_{ij}} = x_i. \tag{23}$$

  It remains to compute $\frac{\partial E}{\partial z_j}$. Since each hidden output z_j influences every output unit,
  $$\frac{\partial E}{\partial z_j} = \sum_k \frac{\partial E}{\partial y_{in,k}}\frac{\partial y_{in,k}}{\partial z_j}. \tag{24}$$
  Now
  $$\frac{\partial E}{\partial y_{in,k}} = \frac{\partial E}{\partial y_k}\frac{\partial y_k}{\partial y_{in,k}} = -(t_k - y_k)f'(y_{in,k}) = -\delta_k, \tag{25}$$
  $$\frac{\partial y_{in,k}}{\partial z_j} = w_{jk}. \tag{26}$$
  Therefore
  $$\frac{\partial E}{\partial z_j} = \sum_k (-\delta_k)w_{jk} = -\sum_k \delta_k w_{jk}. \tag{27}$$
  Substitute into the derivative of E with respect to v_{ij}:
  $$\frac{\partial E}{\partial v_{ij}} = \left(-\sum_k \delta_k w_{jk}\right) f'(z_{in,j})x_i. \tag{28}$$

Define the hidden-layer error term

$$\delta_j = f'(z_{in,j}) \sum_k \delta_k w_{jk}. \tag{29}$$

Then

$$\frac{\partial E}{\partial v_{ij}} = -\delta_j x_i. \tag{30}$$

Applying gradient descent:

$$v_{ij}^{\text{new}} = v_{ij}^{\text{old}} - \alpha \frac{\partial E}{\partial v_{ij}} = v_{ij}^{\text{old}} + \alpha \delta_j x_i. \tag{31}$$

Hence

$$\Delta v_{ij} = \alpha \delta_j x_i. \tag{32}$$

This is exactly the hidden-layer weight update rule in backpropagation.

- **Hidden-Layer Bias Update**

  Since

  $$\frac{\partial z_{in,j}}{\partial v_{0j}} = 1, \tag{33}$$

  we get

  $$\frac{\partial E}{\partial v_{0j}} = \frac{\partial E}{\partial z_j} \frac{\partial z_j}{\partial z_{in,j}} \frac{\partial z_{in,j}}{\partial v_{0j}} = -\delta_j. \tag{34}$$

  Thus

  $$v_{0j}^{\text{new}} = v_{0j}^{\text{old}} + \alpha \delta_j, \tag{35}$$

  so

  $$\Delta v_{0j} = \alpha \delta_j. \tag{36}$$

- Thus, we have shown that:

  $$\Delta w_{jk} = \alpha \delta_k z_j, \tag{37}$$
  $$\Delta w_{0k} = \alpha \delta_k, \tag{38}$$
  $$\Delta v_{ij} = \alpha \delta_j x_i, \tag{39}$$
  $$\Delta v_{0j} = \alpha \delta_j, \tag{40}$$

  where

  $$\delta_k = (t_k - y_k) f'(y_{in,k}), \tag{41}$$
  $$\delta_j = f'(z_{in,j}) \sum_k \delta_k w_{jk}. \tag{42}$$

  Therefore, the backpropagation algorithm is precisely gradient descent applied to the squared-error objective in a multilayer feedforward neural network.

□

1.6.5 Standard Backpropagation Algorithm

- **Step 0: Initialize Weights** Initialize all weights and biases to small random values.
- **Step 1: Repeat Until Stopping Condition Holds** For each training pair $(\mathbf{x}, \mathbf{t})$, do the following.
  - **Step 2: Present input** Apply the input vector

$$\mathbf{x} = (x_1, x_2, \ldots, x_n)$$

    to the input layer.
  - **Step 3: Compute hidden layer outputs** For each hidden unit Z_j,

$$z_{in,j} = v_{0j} + \sum_{i=1}^{n} x_i v_{ij}, \tag{43}$$

$$z_j = f(z_{in,j}). \tag{44}$$

  - **Step 4: Compute output layer outputs** For each output unit Y_k,

$$y_{in,k} = w_{0k} + \sum_{j=1}^{p} z_j w_{jk}, \tag{45}$$

$$y_k = f(y_{in,k}). \tag{46}$$

  - **Step 5: Compute output error terms** For each output unit Y_k, compare the computed output y_k with the target t_k:

$$\delta_k = (t_k - y_k) f'(y_in, k). \tag{47}$$

    Then compute the correction terms:

$$\Delta w_{jk} = \alpha \delta_k z_j, \tag{48}$$

$$\Delta w_{0k} = \alpha \delta_k, \tag{49}$$

    where α is the learning rate.
  - **Step 6: Compute hidden error terms** For each hidden unit Z_j,

$$\delta_{in,j} = \sum_{k=1}^{m} \delta_k w_{jk}, \tag{50}$$

$$\delta_j = \delta_{in,j} f'(z_{in,j}). \tag{51}$$

    Then compute:

$$\Delta v_{ij} = \alpha \delta_j x_i, \tag{52}$$

$$\Delta v_{0j} = \alpha \delta_j. \tag{53}$$

- **Weight Update Phase**

  Update the output-layer parameters:

$$w_{jk}^{\text{new}} = w_{jk}^{\text{old}} + \Delta w_{jk}, \tag{54}$$
$$w_{0k}^{\text{new}} = w_{0k}^{\text{old}} + \Delta w_{0k}. \tag{55}$$

  Update the hidden-layer parameters:

$$v_{ij}^{\text{new}} = v_{ij}^{\text{old}} + \Delta v_{ij}, \tag{56}$$
$$v_{0j}^{\text{new}} = v_{0j}^{\text{old}} + \Delta v_{0j}. \tag{57}$$

1.6.6 Stopping Condition

Training continues until a stopping criterion is satisfied, such as:

- sufficiently small total squared error,
- maximum number of epochs,
- validation error begins to increase.

1.6.7 Important Practical Design Choices

- **Initialization of Weights**: Initial weights should be small random values. If weights are too large, the network may enter sigmoid saturation regions where derivatives are near zero, causing slow learning. If weights are too small, signals may remain too close to zero, also slowing learning [2].

  Before training begins, a neural network's weights must be assigned initial values. The choice of these values strongly affects whether the network learns effectively.

- **Why random nonzero values are needed.** If all weights start at zero, or if all neurons in the same hidden layer start with identical weights, then those neurons will produce identical outputs and receive identical gradients during backpropagation. As a result, they will continue to update in the same way, preventing the network from learning distinct features. This is known as the *symmetry problem.*

- **Risk of large initial weights.** Sigmoid and hyperbolic tangent activation functions map inputs into bounded ranges. When the weighted sum

$$z = \sum_i w_i x_i + b$$

  becomes too large in magnitude, the activation enters the flat outer region of the curve, where

$$f'(z) \approx 0.$$

  During backpropagation, gradients are multiplied by these small derivatives, so the weight updates become negligible. This is called *gradient saturation* or a form of the *vanishing gradient problem.*

- **Risk of very small initial weights.** If the initial weights are extremely close to zero, then signals passing through the layers may remain too small to produce meaningful activations. In deeper networks, both activations and gradients may shrink so much that learning becomes very slow.

Example 4.

1.6.8 Nguyen–Widrow Initialization

Nguyen–Widrow initialization is an improved weight-initialization technique that rescales hidden-layer weight vectors using a factor of the form

$$\beta = 0.7\, p^{1/n},$$

where p is the number of hidden units and n is the number of input units. This helps distribute hidden-unit sensitivity more effectively across the input space and often improves convergence speed.
The Nguyen–Widrow algorithm improves upon simple random initialization, especially for hidden layers that use sigmoidal activation functions.

Core idea. Ordinary random initialization distributes neurons arbitrarily throughout the input space. Nguyen–Widrow rescales the weights so that each hidden neuron begins in an active, nonsaturated region and covers a meaningful part of the input domain.

Interpretation of the formula.

$$\beta = 0.7\, p^{1/n}$$

Here:

- p = number of hidden units,
- n = number of input units.

The term $p^{1/n}$ reflects how densely the hidden units should cover the n-dimensional input space. The constant 0.7 is an empirical factor chosen so that sigmoid activations remain in useful transition regions rather than immediately saturating.

Procedure. For each hidden unit:

1. Initialize the input weight vector with small random values.
2. Compute its Euclidean norm.
3. Normalize the vector so that its magnitude becomes 1.
4. Rescale the normalized vector to magnitude β.
5. Assign a random bias in the interval $[-\beta, \beta]$.

Benefit. This method often reduces training time significantly because it places hidden units in informative regions of the input space from the beginning.

Example 5.

1.6.9 Advantages and Limitations

Advantages

- Can learn complex nonlinear mappings.
- Works for a wide range of supervised learning problems.
- Once trained, produces outputs quickly.
- Can be adapted for applications beyond classification, such as compression.

Limitations

- Training may be slow.
- Convergence depends strongly on initialization.
- Sigmoid saturation can cause very slow learning.
- Poor representation of inputs/targets can hurt performance.
- Overtraining can reduce generalization.

Example 6.

References

1. Goodfellow, I., Bengio, Y., & Courville, A. *Deep Learning*. Chapter on Multilayer Perceptrons. Available at: `https://www.deeplearningbook.org/contents/mlp.html`
2. L. Fausett, *Fundamentals of Neural Networks: Architectures, Algorithms, and Applications*, Prentice Hall, 1994.

1 Variations of Backpropagation

Several modifications can be made to the standard backpropagation algorithm in order to improve its performance in particular situations.

1.1 Momentum

Momentum is a modification of gradient descent in which the weight change depends not only on the current gradient, but also on the previous weight change. Thus, the update direction reflects both the current error signal and the recent history of learning.
For the weights connecting hidden unit j to output unit k, the momentum update rule is

$$w_{jk}(t+1) = w_{jk}(t) + \alpha\delta_k z_j + \mu\left[w_{jk}(t) - w_{jk}(t-1)\right],$$

or equivalently,

$$\Delta w_{jk}(t+1) = \alpha\delta_k z_j + \mu\Delta w_{jk}(t).$$

For the weights connecting input unit i to hidden unit j, the corresponding rule is

$$v_{ij}(t+1) = v_{ij}(t) + \alpha\delta_j x_i + \mu\left[v_{ij}(t) - v_{ij}(t-1)\right],$$

or equivalently,

$$\Delta v_{ij}(t+1) = \alpha\delta_j x_i + \mu\Delta v_{ij}(t).$$

In these expressions:

- t denotes the current training step,
- α is the learning rate,
- μ is the momentum parameter,
- δ_k is the error term for output unit k,
- δ_j is the error term for hidden unit j,
- z_j is the output of hidden unit j,
- x_i is the input from input unit i,
- $\Delta w_{jk}(t)$ and $\Delta v_{ij}(t)$ are the previous weight changes.

The momentum parameter is constrained by

$$0 < \mu < 1.$$

Purpose. Momentum smooths the learning trajectory:

- it accelerates learning when gradients keep pointing in similar directions,
- it reduces sensitivity to isolated noisy training examples,
- it may help the network move through shallow local irregularities.

Example 1.

1.2 Batch Updating

In standard backpropagation, weights are often updated immediately after each training pattern is presented. An alternative is *batch updating*, in which weight corrections are accumulated over several training patterns, or even over an entire epoch, before the actual update is applied.
The main idea is that instead of changing a weight after every single example, the network first computes the corrections for many examples and then combines them into one overall adjustment. If the corrections for a weight from P training patterns are denoted by

$$\Delta w^{(1)}, \Delta w^{(2)}, \ldots, \Delta w^{(P)},$$

then a batch update may be written as

$$\Delta w^{\text{batch}} = \sum_{r=1}^{P} \Delta w^{(r)},$$

or, in averaged form,

$$\Delta w^{\text{avg}} = \frac{1}{P} \sum_{r=1}^{P} \Delta w^{(r)}.$$

Instead of updating weights after each training pattern, batch updating accumulates corrections over multiple patterns, often over one full epoch, and then applies the average or sum:

$$\Delta w_{jk}^{\text{batch}} = \sum_{r=1}^{P} \Delta w_{jk}^{(r)}.$$

Purpose. Batch updating can be more stable because it reduces the influence of any single training example.

Example 2.

1.3 Adaptive Learning Rates

A fixed learning rate is not always optimal throughout training. Some weights may require large updates, while others may require small and careful adjustments. For this reason, adaptive learning-rate methods were introduced to allow the effective step size to change during learning. One important example is *delta-bar-delta*.

1.3.1 Delta-Bar-Delta (DBD) Algorithm

Standard backpropagation uses a single global learning rate, creating a trade-off: a rate that is too small slows convergence, while a rate that is too large causes oscillation or divergence. Delta-Bar-Delta overcomes this by assigning an independent, time-varying learning rate $\alpha_i(t)$ to each weight w_i.

Core Intuition

- **Same gradient sign across iterations:** The parameter is moving consistently along a gentle slope → **increase** $\alpha_i(t)$.
- **Alternating gradient sign:** The parameter is overshooting a minimum and bouncing across a valley → **decrease** $\alpha_i(t)$.

Mathematical Formulation

1. **Exponentially Smoothed Gradient:** To mitigate gradient noise, DBD maintains a running average $\bar{\delta}_i(t)$ of past gradients $g_i(t) = \dfrac{\partial E(t)}{\partial w_i}$:

$$\bar{\delta}_i(t) = (1-\theta)\, g_i(t) + \theta\, \bar{\delta}_i(t-1), \qquad 0 < \theta < 1. \tag{1}$$

2. **Learning-Rate Adaptation Rule:** The update direction is determined by the sign of the product $\bar{\delta}_i(t-1)\, g_i(t)$:

$$\Delta\alpha_i(t) = \begin{cases} \kappa, & \text{if } \bar{\delta}_i(t-1)\, g_i(t) > 0, \\ -\phi\, \alpha_i(t-1), & \text{if } \bar{\delta}_i(t-1)\, g_i(t) < 0, \\ 0, & \text{otherwise,} \end{cases} \tag{2}$$

$$\alpha_i(t) = \alpha_i(t-1) + \Delta\alpha_i(t), \tag{3}$$

where $\kappa > 0$ is an additive increment parameter and $0 < \phi < 1$ controls the multiplicative decrease.

> **Note on Asymmetry:** Increments are *linear/additive* (κ) to avoid sudden explosive step sizes, while decrements are *exponential/multiplicative* ($-\phi\alpha_i$) to rapidly damp oscillations near minima.

3. **Weight Update:** Using the newly adapted local step size, the weight is updated via standard gradient descent:

$$w_i(t+1) = w_i(t) - \alpha_i(t)\, g_i(t). \tag{4}$$

Comparison: Advantages vs. Limitations -

Table 1: Key properties of the Delta-Bar-Delta algorithm.

Advantages	**Limitations**
Accelerates convergence along flat plateau regions.	Introduces sensitive hyperparameters (κ, ϕ, θ).
Damps oscillations across steep ravines/valleys.	Lacks convergence guarantees on highly non-convex surfaces.
Provides parameter-wise local adaptation.	Paves the way for modern methods (Rprop, RMSprop, Adam).

Example 3.

1.4 Alternative Activation Functions

Standard backpropagation uses fixed, bounded activation functions such as the logistic sigmoid. Modifying the activation function directly alters the gradient dynamics, the output bounds, and the effective capacity of the network *without requiring any change to the underlying optimization algorithm.* Below are few key variations analyzed with full derivations and practical explanations.

1. **Customized (Scaled and Shifted) Sigmoids**

   Standard backprop assumes targets in $[0, 1]$ (logistic output) or $[-1, 1]$ (bipolar output). For arbitrary target intervals $[a, b]$, building the bounds into the activation function avoids artificial output rescaling and prevents severe output-layer saturation near range boundaries:

$$f(x) = a + (b - a)\,\sigma(x) = a + \frac{b - a}{1 + e^{-x}} \tag{5}$$

   Step-by-Step Derivation

   Differentiating Eq. (5) using $\sigma'(x) = \sigma(x)(1 - \sigma(x))$:

$$\frac{\partial f}{\partial x} = (b - a) \cdot \sigma(x)\big(1 - \sigma(x)\big) \tag{6}$$

   Inverting Eq. (5) to express the derivative in terms of the unit's output $y = f(x)$:

$$\sigma(x) = \frac{y - a}{b - a} \tag{7}$$

   Substituting Eq. (7) into Eq. (6) simplifies cleanly:

$$\frac{\partial f}{\partial x} = (b - a)\left(\frac{y - a}{b - a}\right)\left(1 - \frac{y - a}{b - a}\right) = \frac{(y - a)(b - y)}{b - a} \tag{8}$$

   Takeaway: The derivative is purely a function of the output y. It peaks at $(b-a)/4$ at the midpoint $y = (a+b)/2$ and vanishes quadratically at bounds $y \to a$ or $y \to b$.

   Special cases: Setting $[a, b] = [0, 1]$ yields $y(1 - y)$; setting $[a, b] = [-1, 1]$ yields $\dfrac{1 - y^2}{2}$ (the bipolar sigmoid).

2. **Slope-Adjusted Sigmoid**

   Introducing a fixed gain parameter $\sigma > 0$ controls activation steepness:

$$f(x) = \frac{1}{1 + e^{-\sigma x}}, \qquad \frac{\partial f}{\partial x} = \sigma f(x)\big(1 - f(x)\big) \tag{9}$$

   Behavior & Gradient Scaling

   - **Asymptotics:** As $\sigma \to \infty$, $f(x)$ becomes a step function (sharp decision boundary). As $\sigma \to 0$, $f(x) \to 0.5$ (flat line).
   - **Gradient Multiplier:** The parameter σ acts as a direct scale factor on local error deltas ($\delta_j \propto \sigma$).

- **Effective Step Size:** Since both forward gain and backward deltas scale with σ, weight updates scale as $\eta_w \sigma^2$.

3. **Adaptive Slopes (Learnable Gain Parameters)**

   Each neuron j maintains an independent, learnable slope σ_j to self-tune sensitivity during training.

   Derivation of Updates

   Let $z_j = \sum_i w_{ji} x_i + b_j$ and output $a_j = f(z_j; \sigma_j) = \dfrac{1}{1 + e^{-\sigma_j z_j}}$:

$$a_j = \frac{1}{1 + e^{-\sigma_j z_j}}, \qquad \delta_j = \frac{\partial E}{\partial a_j} \cdot \sigma_j a_j (1 - a_j) \tag{10}$$

$$\frac{\partial E}{\partial w_{ji}} = \delta_j x_i, \qquad \frac{\partial f}{\partial \sigma_j} = z_j a_j (1 - a_j) \tag{11}$$

   Applying the chain rule to derive the slope gradient yields an elegant cancellation:

$$\frac{\partial E}{\partial \sigma_j} = \frac{\partial E}{\partial a_j} \cdot z_j a_j (1 - a_j) = \frac{z_j \delta_j}{\sigma_j} \tag{12}$$

   The parameter update rules are:

$$\Delta w_{ji} = -\eta_w \delta_j x_i \tag{13}$$

$$\Delta \sigma_j = -\eta_\sigma \frac{z_j \delta_j}{\sigma_j} \tag{14}$$

   Practical Insights & Precautions

   - **Mechanism:** Slopes steepen most for units with large net inputs z_j alongside large error signals δ_j.
   - **Numerical Stability:** The $1/\sigma_j$ term can cause instability if $\sigma_j \to 0$. Constrain $\sigma_j > 0$ via re-parameterization ($\sigma_j = e^{s_j}$) or clipping, and set $\eta_\sigma \ll \eta_w$.
   - **Convergence Speed:** While σ_j can be absorbed into weights, learning a single gain parameter per unit acts as an adaptive learning rate, significantly accelerating convergence.

4. **Other Activation Functions**

1.5 Number of Hidden Layers

Although a single hidden layer is theoretically sufficient to solve any function approximation problem, some problems may be easier to solve using a network with two hidden layers.
The main idea is that a two-hidden-layer architecture can organize the classification task in stages. In such a network:

- the **first hidden layer** often partitions the input space into regions,

Function	$f(x)$	**Range**	$f'(x)$	**Key Property & Backward Pass Impact**
Bipolar Sigmoid	$\frac{1-e^{-\sigma x}}{1+e^{-\sigma x}}$	$(-1, 1)$	$\frac{\sigma}{2}\big(1-f(x)^2\big)$	Zero-centered outputs prevent directional update bias.
Arctangent	$\arctan(x)$	$\big(-\frac{\pi}{2}, \frac{\pi}{2}\big)$	$\frac{1}{1+x^2}$	Polynomial tail decay ($\sim 1/x^2$) preserves stronger gradients for $\lvert x\rvert \gg 1$.
Identity	x	$(-\infty, \infty)$	1	Linear output for regression; avoids output saturation entirely.
Gaussian (RBF)	$\exp\left(-\frac{\lVert x-c\rVert^2}{2r^2}\right)$	$(0, 1]$	$-\frac{x-c}{r^2}f(x)$	Localized activations; unit responds to input regions rather than hyperplanes.

- the **second hidden layer** represents clusters of points within those regions,
- the **output layer** performs the final classification if those clusters are separable.

Thus, while one hidden layer is sufficient in theory, two hidden layers may offer a more convenient internal structure for certain classification problems.
The excerpt also notes a sample-complexity style result for classification of M clusters. In this setting, the number of boundary samples is of the order

$$\min(n, p) \cdot M,$$

where:

- n is the dimension of the input space, that is, the number of input units,
- p is the number of hidden nodes,
- M is the number of clusters to be classified.

This result suggests that for some structured classification tasks, two-hidden-layer networks may provide a more efficient way to represent cluster boundaries.

1.5.1 Backpropagation with Two Hidden Layers

Although one hidden layer is theoretically sufficient for approximating any continuous mapping, some problems are easier to solve with two hidden layers.
Let:

- first hidden layer: z_j,
- second hidden layer: u_h,
- output layer: y_k.

Feedforward equations:

$$z_{in,j} = v_{0j} + \sum_i x_i v_{ij}, \qquad z_j = f(z_{in,j}),$$

$$u_{in,h} = u_{0h} + \sum_j z_j u_{jh}, \qquad u_h = f(u_{in,h}),$$

$$y_{in,k} = w_{0k} + \sum_h u_h w_{hk}, \qquad y_k = f(y_{in,k}).$$

Backward error terms:

$$\delta_k = (t_k - y_k) f'(y_{in,k}),$$

$$\delta_h = f'(u_{in,h}) \sum_k \delta_k w_{hk},$$

$$\delta_j = f'(z_{in,j}) \sum_h \delta_h u_{jh}.$$

Weight updates:

$$\Delta w_{hk} = \alpha \delta_k u_h, \qquad \Delta u_{jh} = \alpha \delta_h z_j, \qquad \Delta v_{ij} = \alpha \delta_j x_i.$$

Example 4.

Probabilistic Neural Networks (PNNs)

1.1 What is a Probabilistic Neural Network?

A Probabilistic Neural Network (PNN) is a feedforward classifier based on **Bayesian decision theory** and **nonparametric density estimation**. Instead of learning a small set of weights that summarize the data, a PNN stores training samples and uses them directly to estimate the probability density of each class. Classification is then performed by comparing those estimated class densities. This makes the PNN conceptually close to Parzen-window / kernel-density classification, but implemented as a neural network architecture.

1.2 Main Idea

Suppose we have classes $C_1, C_2, \ldots, C_K$ and an input vector x. For each class C_k, the PNN estimates a class-conditional density

$$f_k(x).$$

Then the network applies the Bayes rule and chooses the class with the largest value of

$$P(C_k)\, L_k\, f_k(x),$$

where

- $P(C_k)$ is the prior probability of class C_k,
- L_k represents a loss/cost factor,
- $f_k(x)$ is the estimated density of class C_k at input x.

If priors and costs are equal, classification reduces to:

$$\hat{y} = \arg\max_k f_k(x).$$

1.3 Architecture of a PNN

A standard PNN has four layers:

1. **Input Layer**
   Receives the feature vector

$$x = [x_1, x_2, \ldots, x_d]^T.$$

2. **Pattern Layer**
   Contains one neuron for each training sample. Each neuron stores one exemplar x_{kj}, where k is the class and j indexes the training sample within that class.

3. **Summation Layer**
   Contains one neuron per class. It sums the activations of all pattern units belonging to that class to estimate the class density.

4. **Output / Decision Layer**
   Compares the class scores and outputs the predicted class.

1.4 Pattern Layer Computation

Each pattern neuron computes a Gaussian kernel centered at its stored training vector. For a training sample x_{kj}, the activation is

$$\phi_{kj}(x) = \frac{1}{(2\pi)^{d/2}\sigma^d} \exp\left(-\frac{\|x - x_{kj}\|^2}{2\sigma^2}\right).$$

Here:

- d is the input dimension,
- σ is the smoothing parameter (bandwidth),
- $\|x - x_{kj}\|^2$ is the squared Euclidean distance between the input and the stored exemplar.

Thus, each training example contributes a local Gaussian "bump" around itself.

1.5 Summation Layer Estimation

If class C_k contains N_k training samples, then the estimated density is

$$\hat{f}_k(x) = \frac{1}{N_k} \sum_{j=1}^{N_k} \phi_{kj}(x).$$

So the class density is simply the average response of all pattern neurons associated with that class.

1.6 Bayes Decision Rule

For the two-class case A and B, the textbook extract gives the decision rule: classify x as belonging to class A if

$$h_A C_A f_A(x) > h_B C_B f_B(x),$$

where:

- h_A, h_B: prior probabilities,
- C_A, C_B: misclassification costs,
- $f_A(x), f_B(x)$: class density estimates.

The decision boundary is therefore determined by

$$h_A C_A f_A(x) = h_B C_B f_B(x).$$

1.7 How Training Works

Training in a PNN is extremely simple compared with backpropagation-based models.

Step 1: Present a training sample.

Step 2: Create one pattern neuron for that sample.

Step 3: Store the sample as that neuron's weight vector.

Step 4: Connect the neuron to the summation unit of its class.

So training is essentially **memory-based**: the network stores examples rather than iteratively optimizing many parameters.

1.8 Why PNNs are Attractive

The main strengths of a PNN are:

- **Instant or near-instant training:** adding a sample is easy.
- **Strong probabilistic interpretation:** outputs are tied to class-density estimates.
- **Asymptotic Bayes optimality:** with enough data and a suitable smoothing factor, the estimator approaches the true class density.
- **Robustness to distant outliers:** Gaussian kernels reduce the influence of far-away points.

1.9 Limitations

The main disadvantages are:

- **High memory cost:** one pattern neuron per training example.
- **Slow inference for large datasets:** prediction cost grows with the number of stored samples.
- **Sensitivity to σ:** if σ is too small, the model overfits; if too large, the model oversmooths.

Example 1.1.